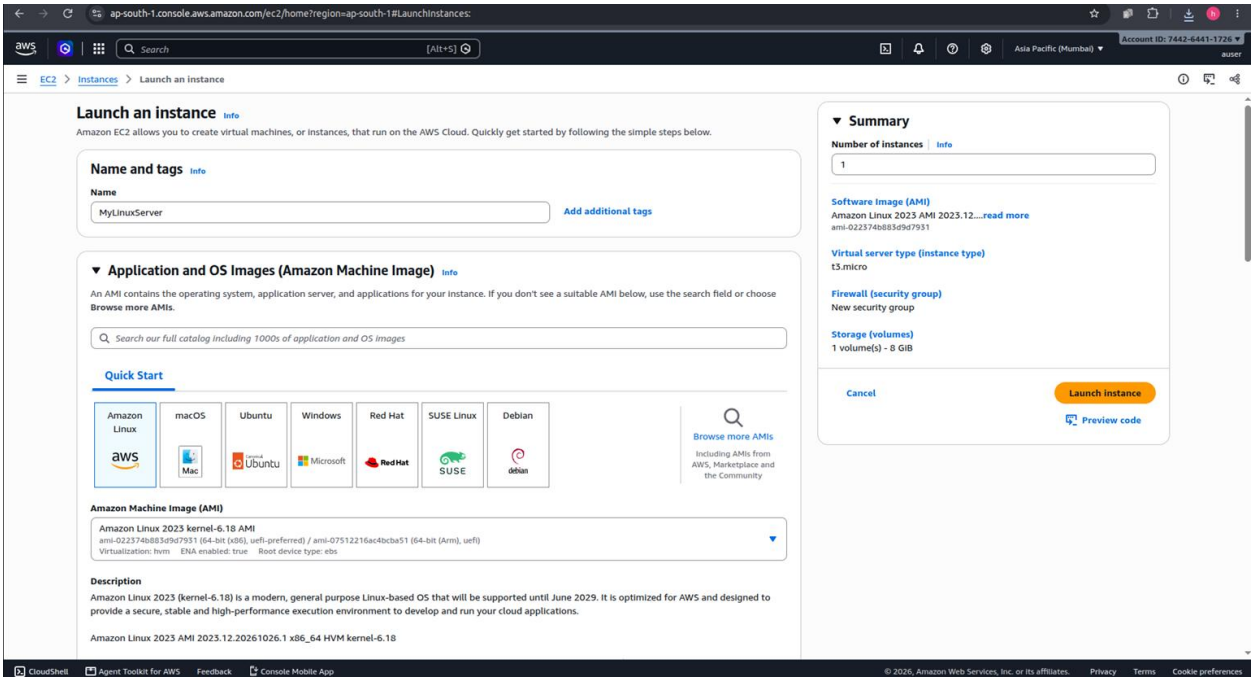
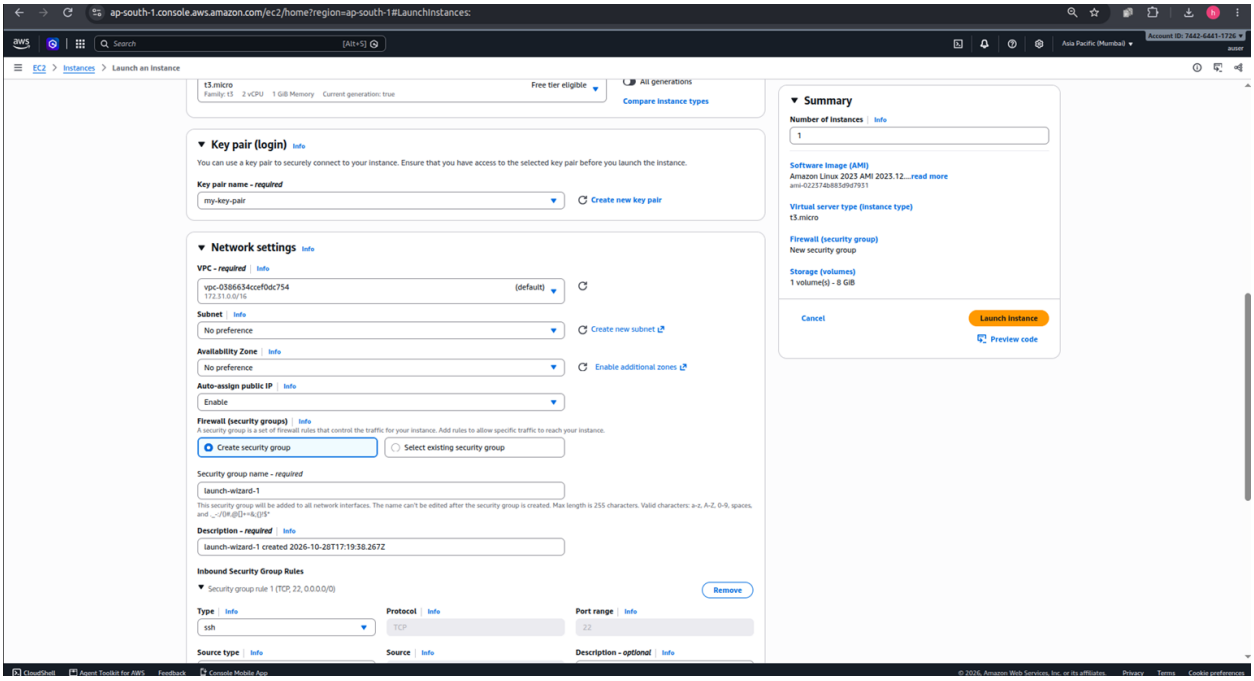
Assignment 7 (Cloud Services & Security)

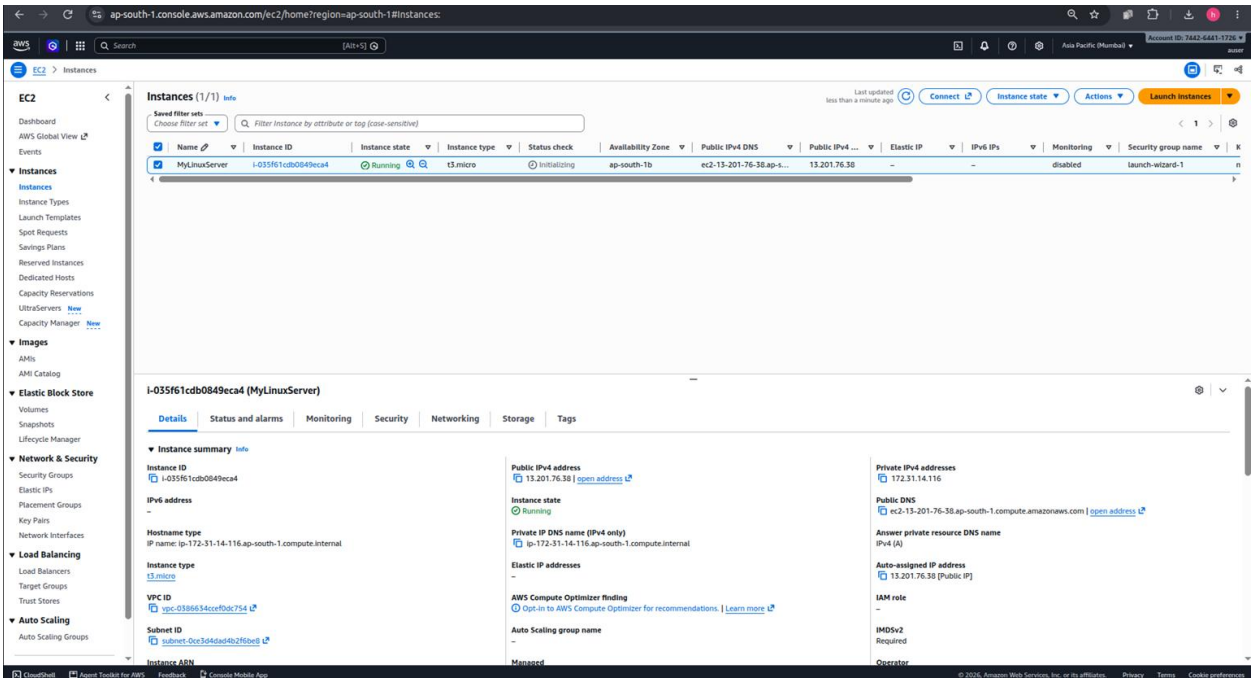
PRN No. - 260840127015 Name - Khendake Pradip Goraksh Course - PGCP-HPCSA

AWS EC2, S3 and Lambda

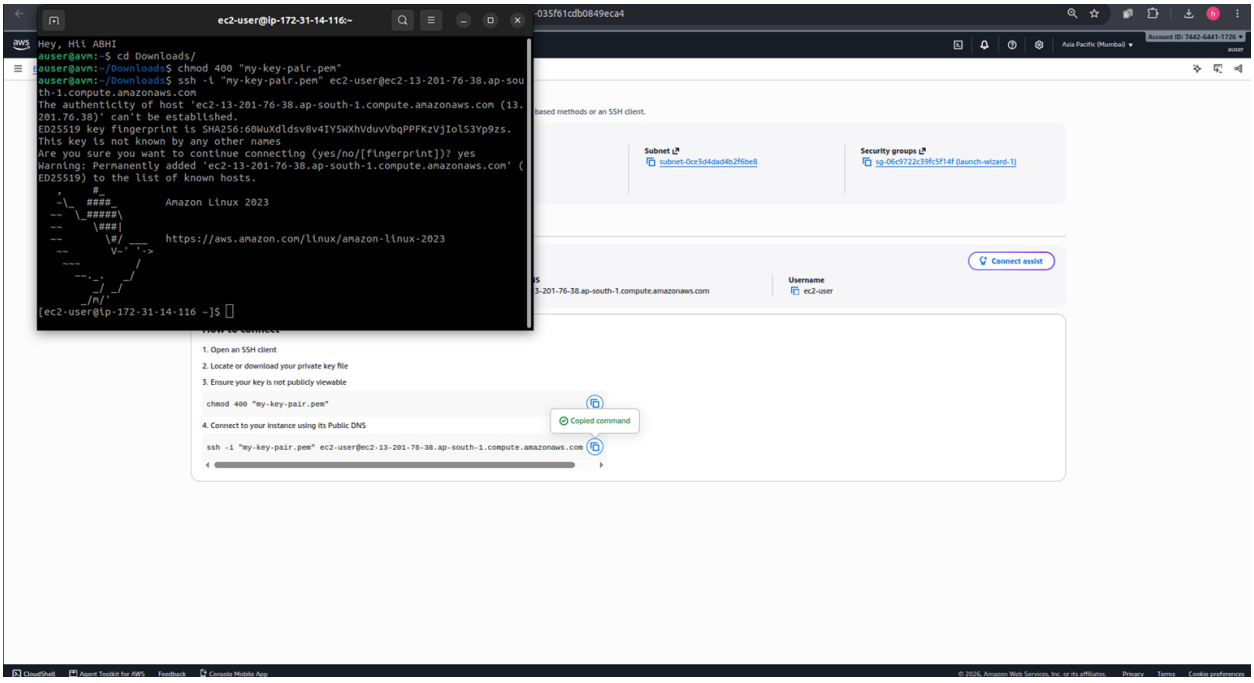
1. Launch an EC2 instance using Amazon Linux and check its status.



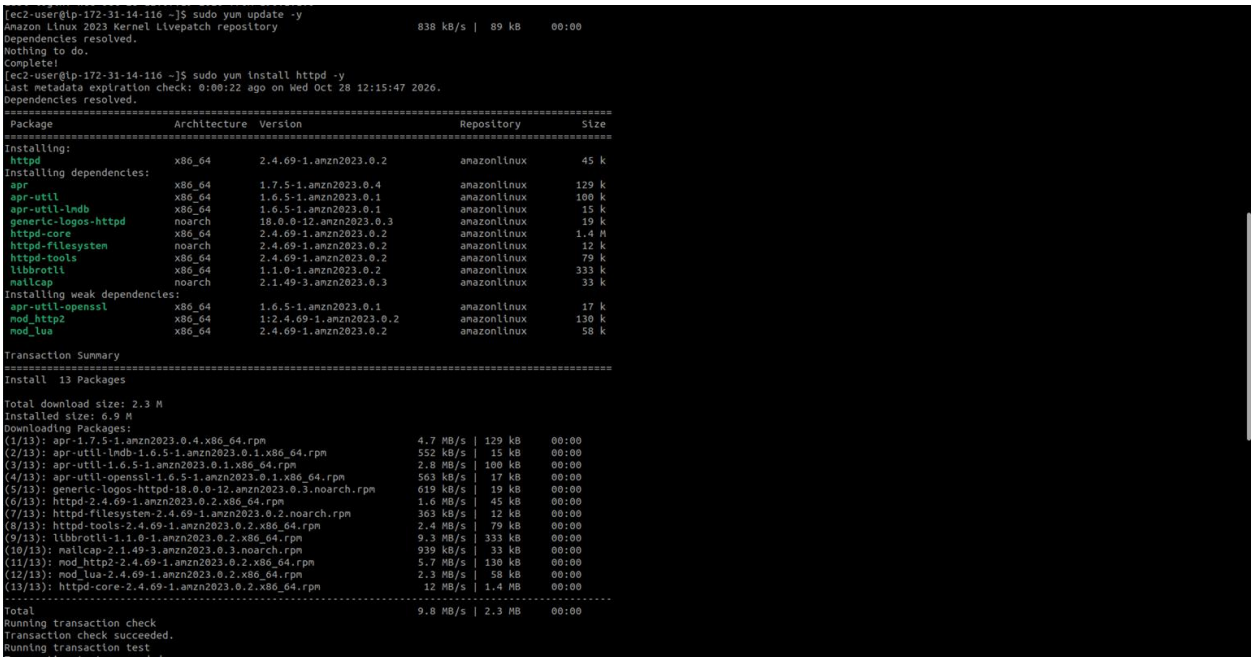


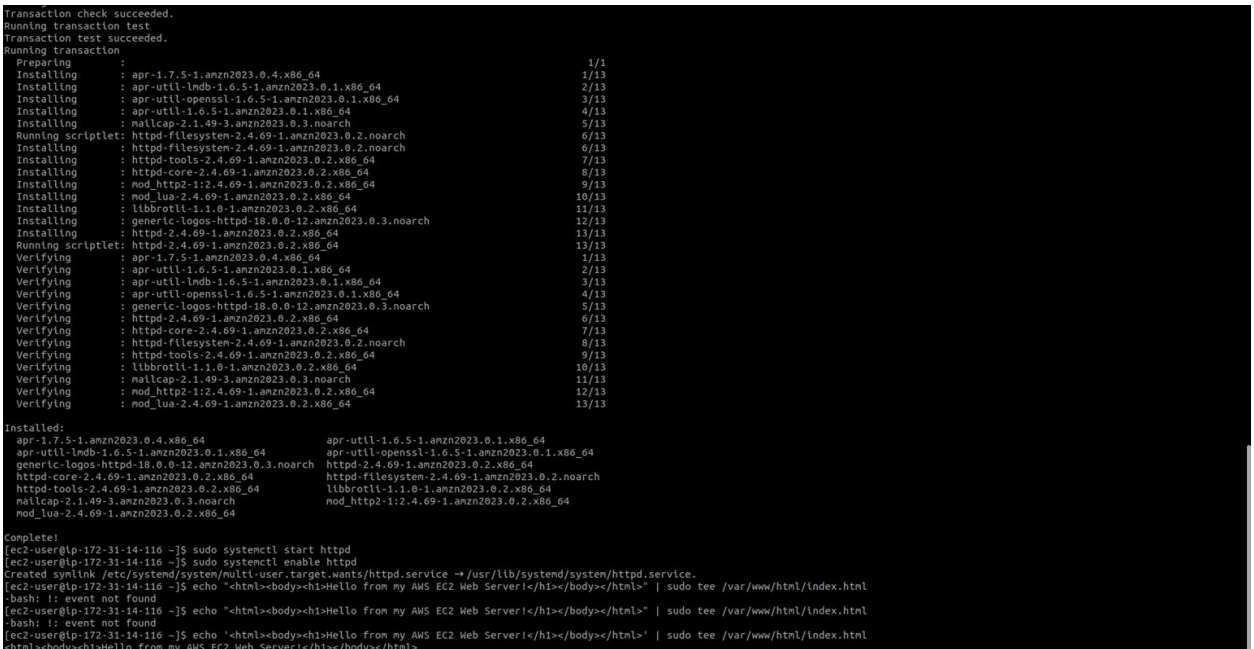


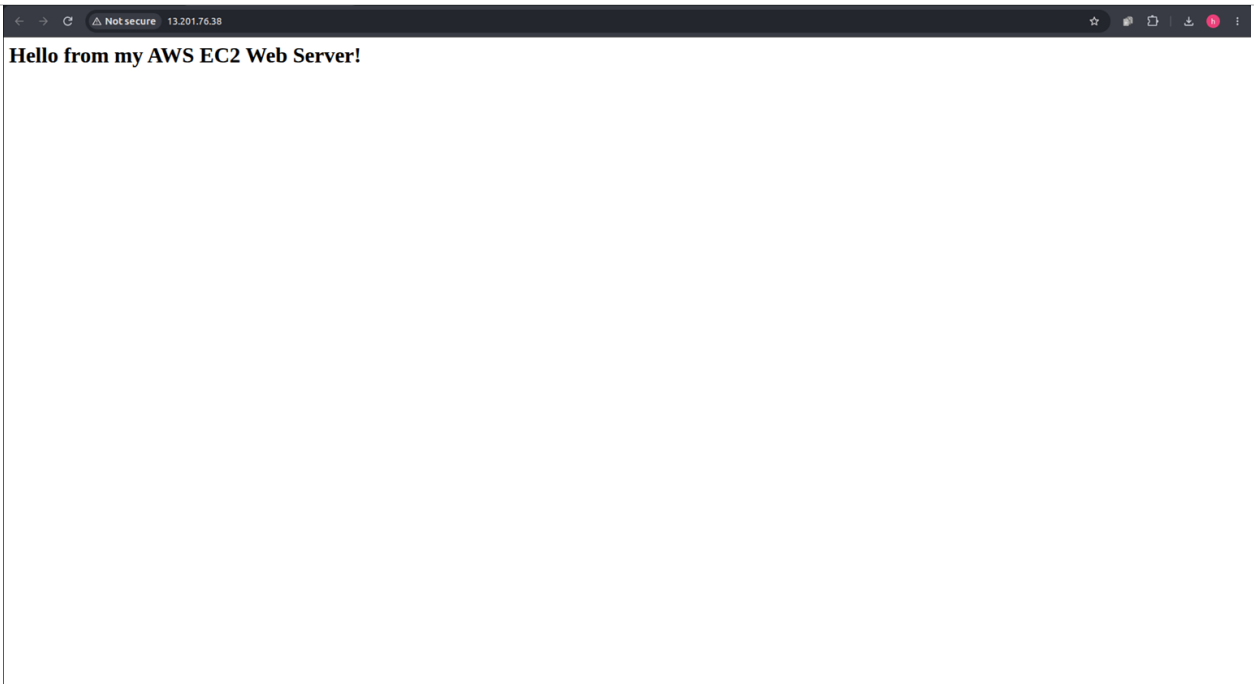
2. Connect to an EC2 instance using SSH or EC2 Instance Connect.



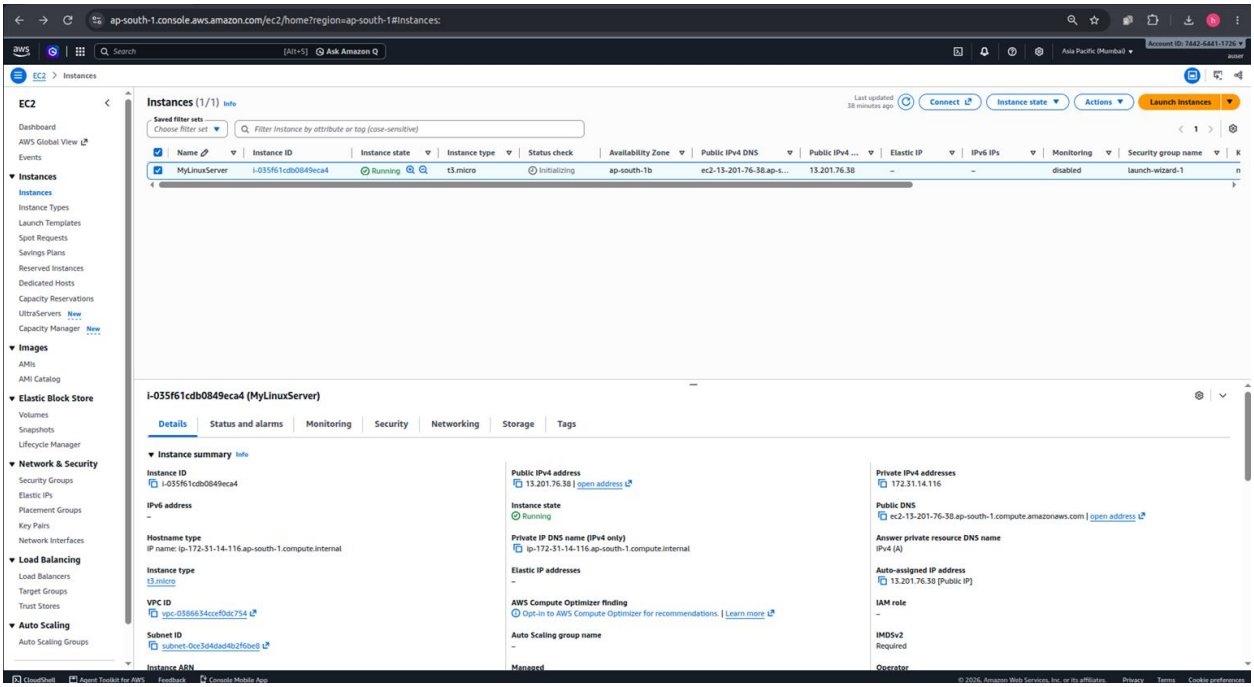
3. Install a web server on an EC2 instance and display a simple HTML page.

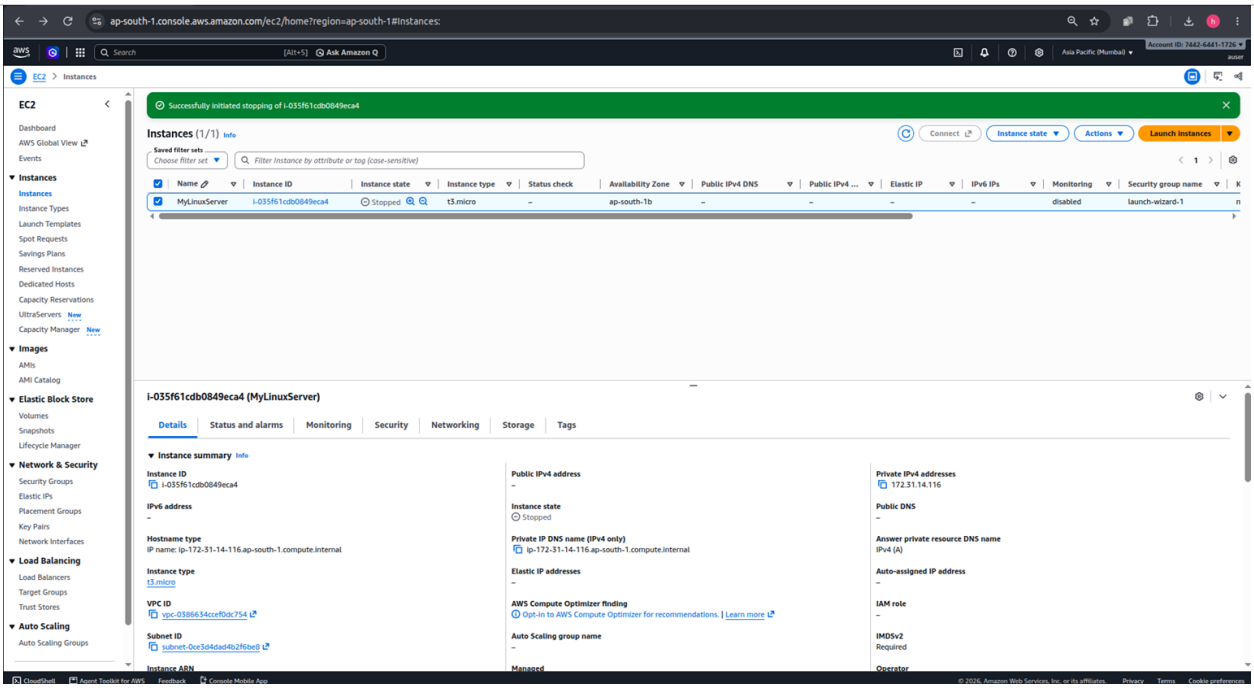


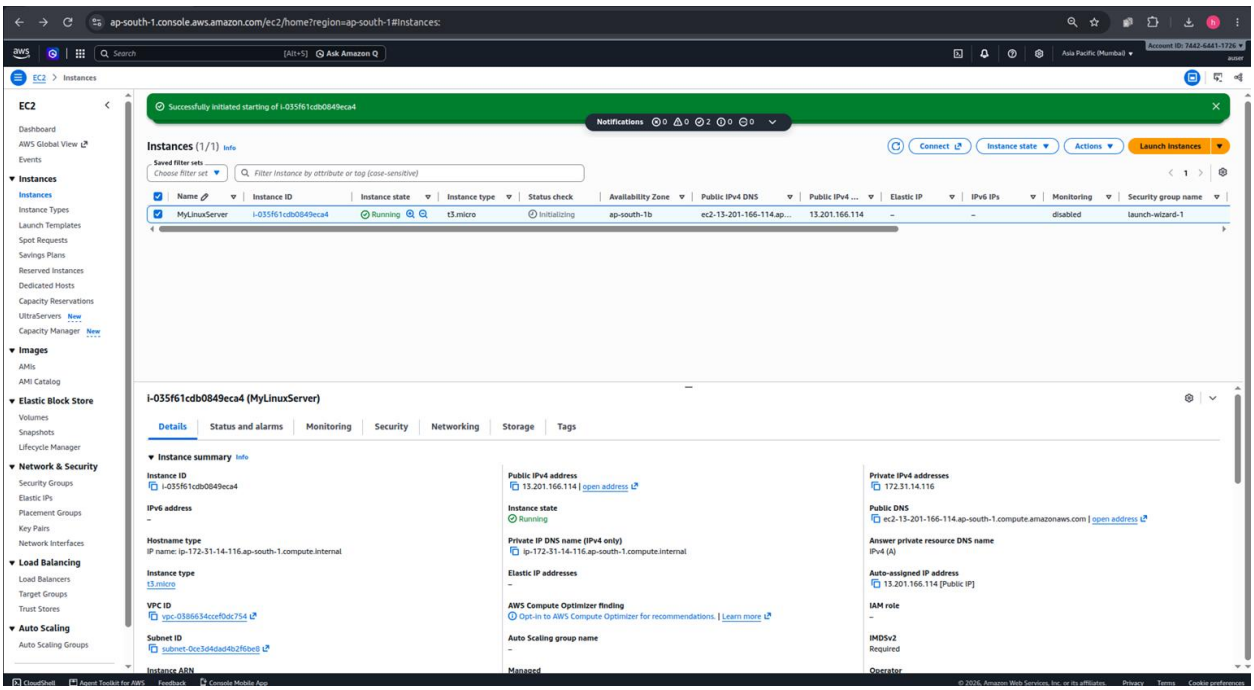




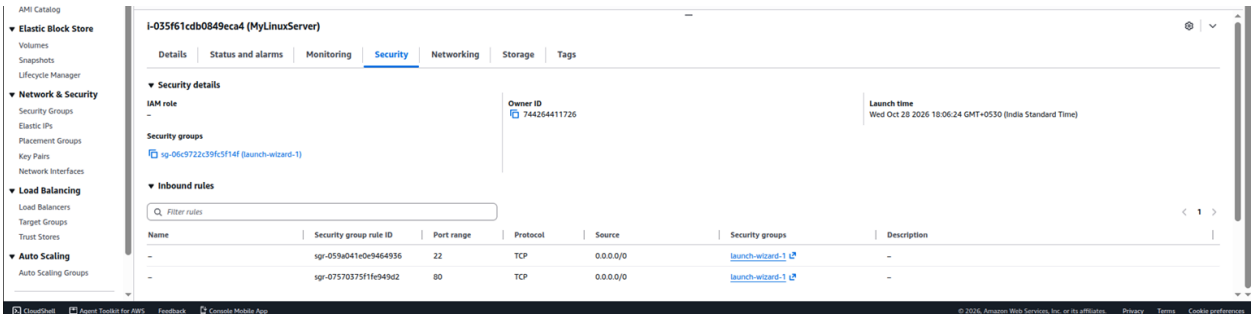
4. Stop and start an EC2 instance and observe the change in its status.

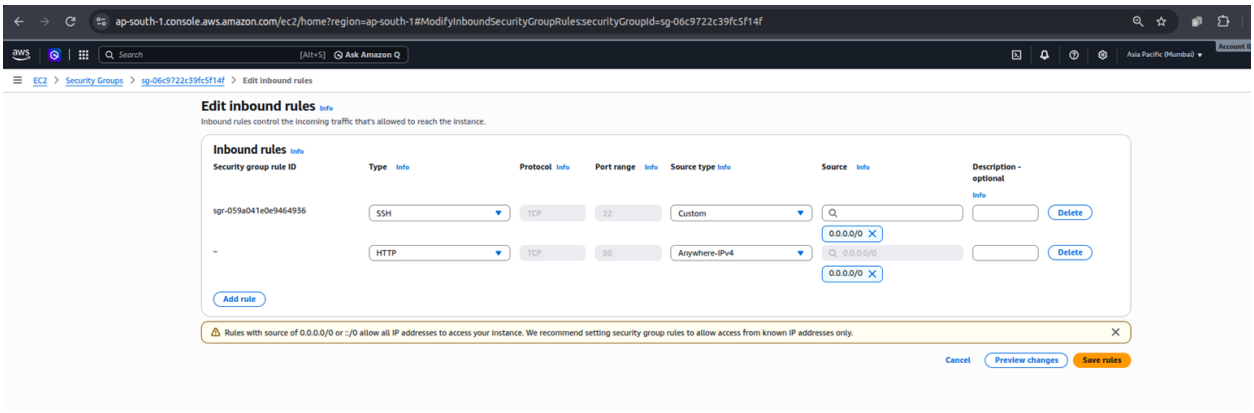



ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#Instances:
Successfully initiated stopping of i-035f61cdb0849eca4
Instances (1/1)
MyLinuxServer
i-035f61cdb0849eca4
Stopped
t3.micro
ap-south-1b
disabled
launch-wizard-1
i-035f61cdb0849eca4 (MyLinuxServer)
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Instance summary
Instance ID
i-035f61cdb0849eca4
IPv6 address
Hostname type
IP name: ip-172-31-14-116.ap-south-1.compute.internal
Instance type
t3.micro
VPC ID
vpc-0386634ccef0dc754
Subnet ID
subnet-0ce3d4dad4b2f6be8
Public IPv4 address
Instance state
Stopped
Private IP DNS name (IPv4 only)
ip-172-31-14-116.ap-south-1.compute.internal
Elastic IP addresses
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto Scaling group name
Private IPv4 addresses
172.31.14.116
Public DNS
Answer private resource DNS name
IPv4 (A)
Auto-assigned IP address
IAM role
IMDSv2
Required

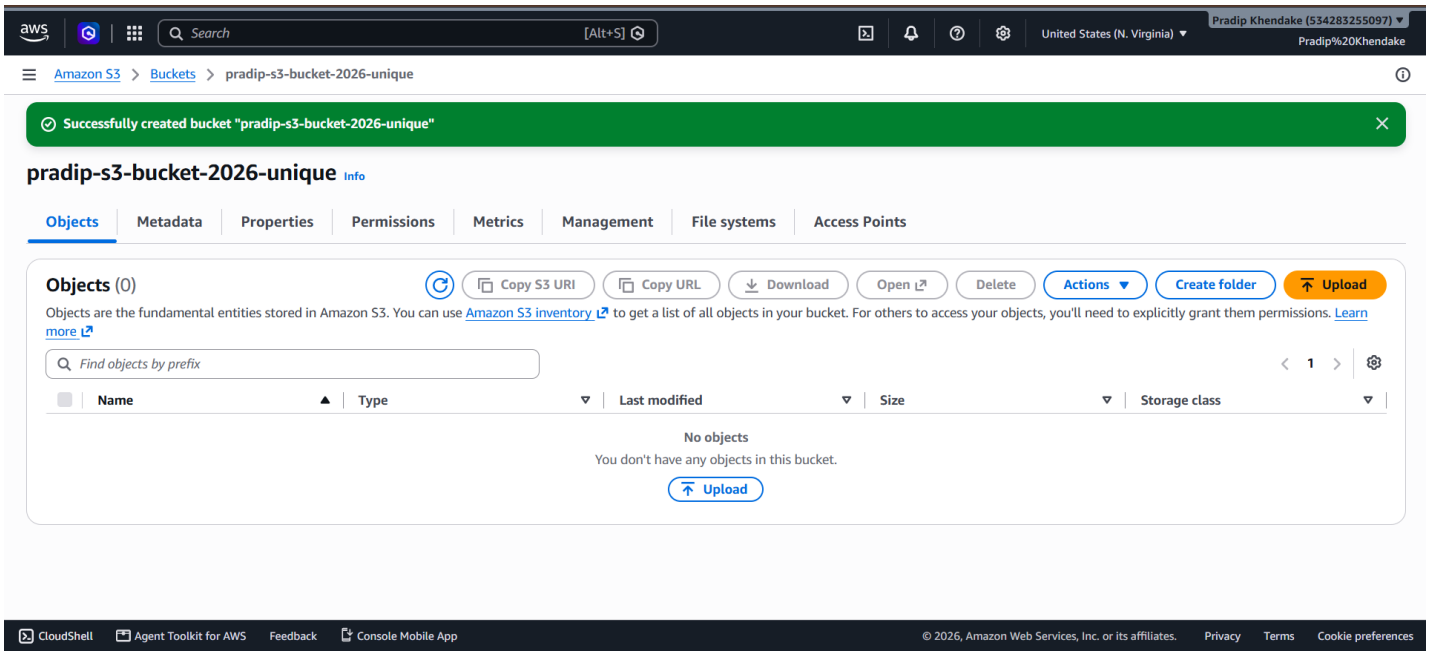

ap-south-1.console.aws.amazon.com/ec2/home?region=ap-south-1#Instances:
Successfully initiated starting of i-035f61cdb0849eca4
Instances (1/1)
MyLinuxServer
i-035f61cdb0849eca4
Running
t3.micro
Initializing
ap-south-1b
ec2-13-201-166-114.ap...
13.201.166.114
disabled
launch-wizard-1
i-035f61cdb0849eca4 (MyLinuxServer)
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Instance summary
Instance ID
i-035f61cdb0849eca4
IPv6 address
Hostname type
IP name: ip-172-31-14-116.ap-south-1.compute.internal
Instance type
t3.micro
VPC ID
vpc-0386634ccef0dc754
Subnet ID
subnet-0ce3d4dad4b2f6be8
Public IPv4 address
13.201.166.114 | open address
Instance state
Running
Private IP DNS name (IPv4 only)
ip-172-31-14-116.ap-south-1.compute.internal
Elastic IP addresses
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations. Learn more
Auto Scaling group name
Private IPv4 addresses
172.31.14.116
Public DNS
ec2-13-201-166-114.ap-south-1.compute.amazonaws.com | open address
Answer private resource DNS name
IPv4 (A)
Auto-assigned IP address
13.201.166.114 [Public IP]
IAM role
IMDSv2
Required

5. Create a security group that allows HTTP traffic on port 80.

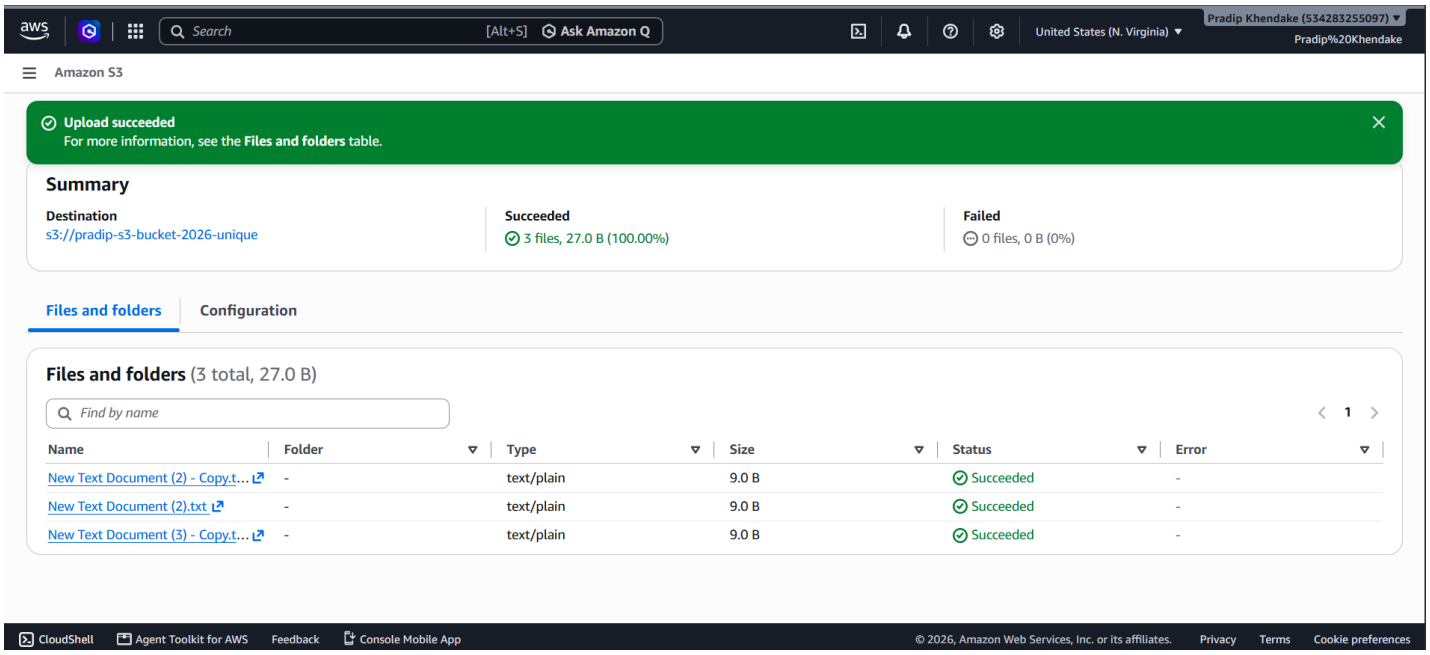


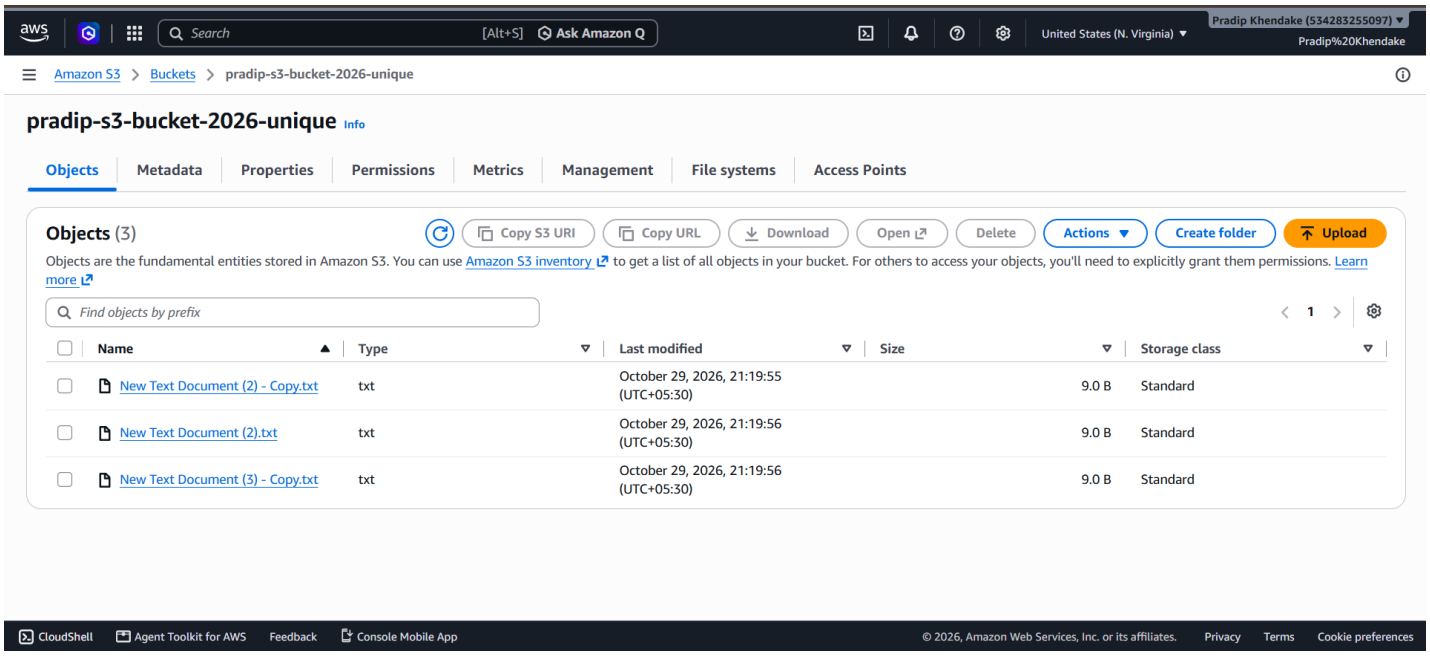


6. Create an S3 bucket with a unique name.

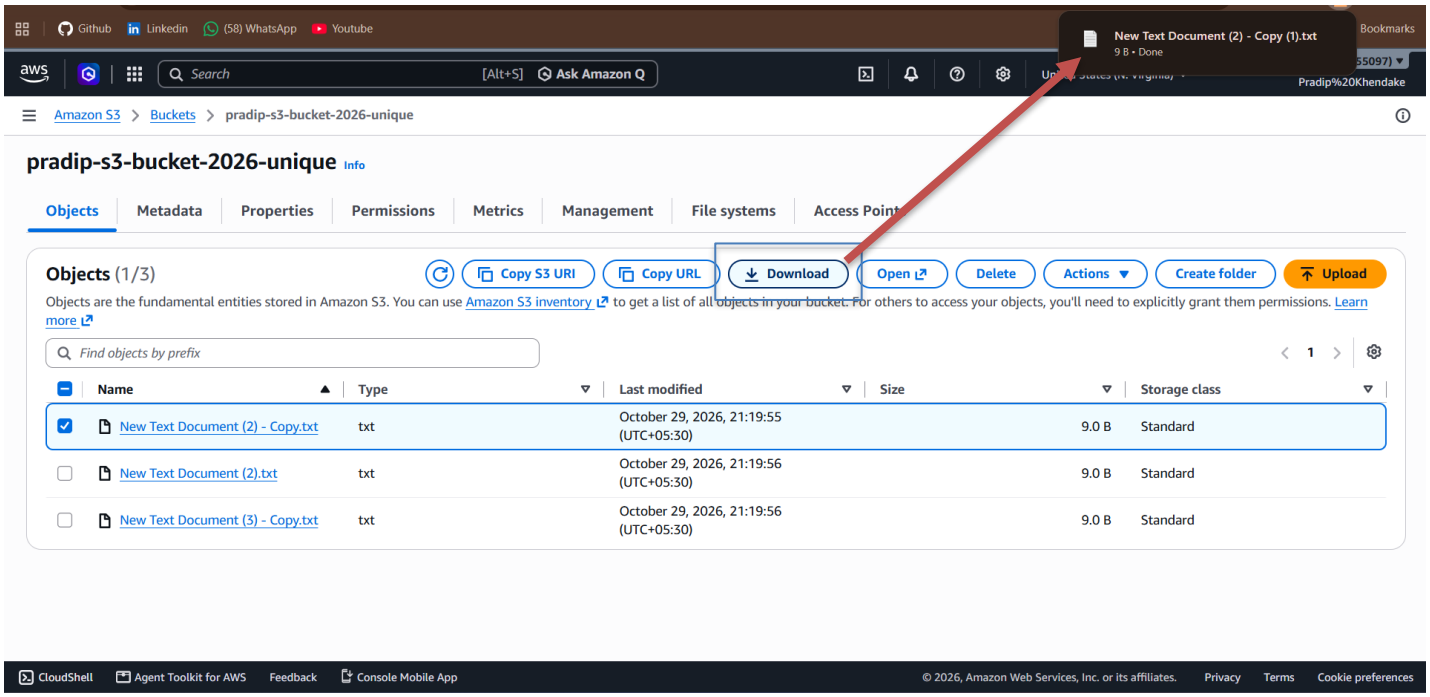


7. Upload three files to an S3 bucket and list the uploaded objects.

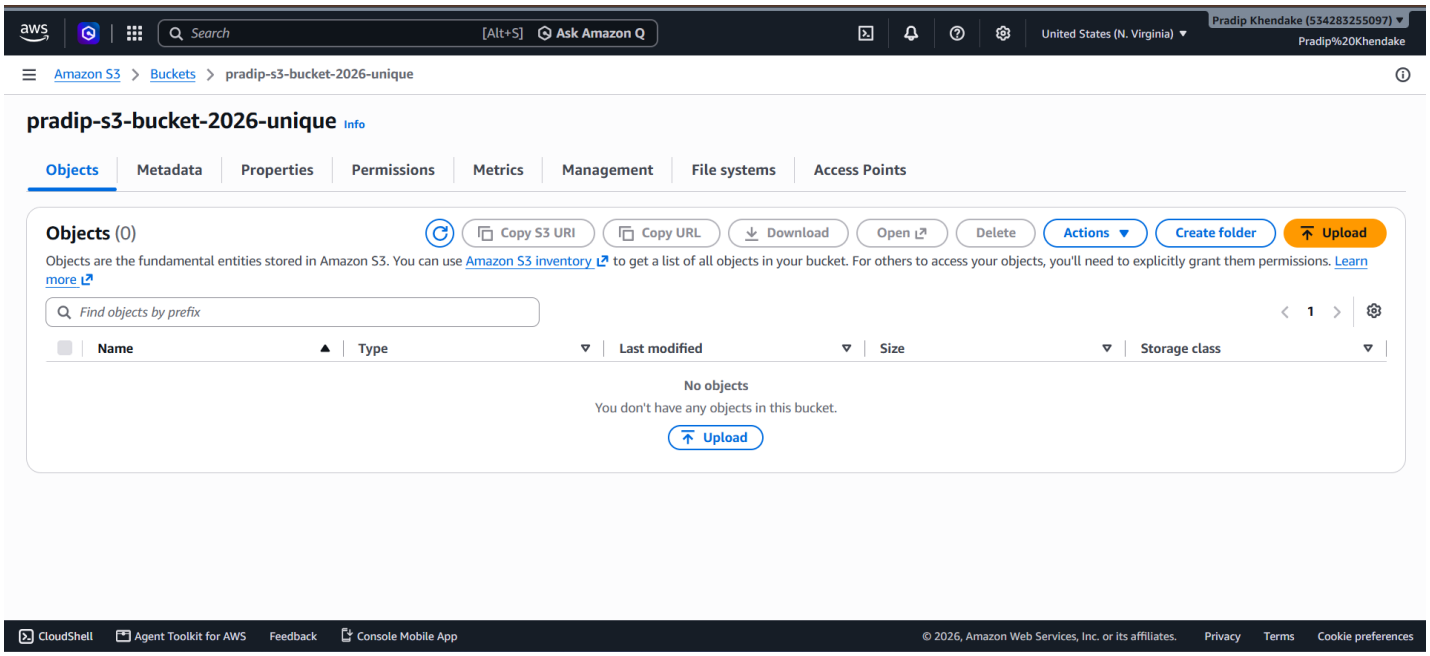


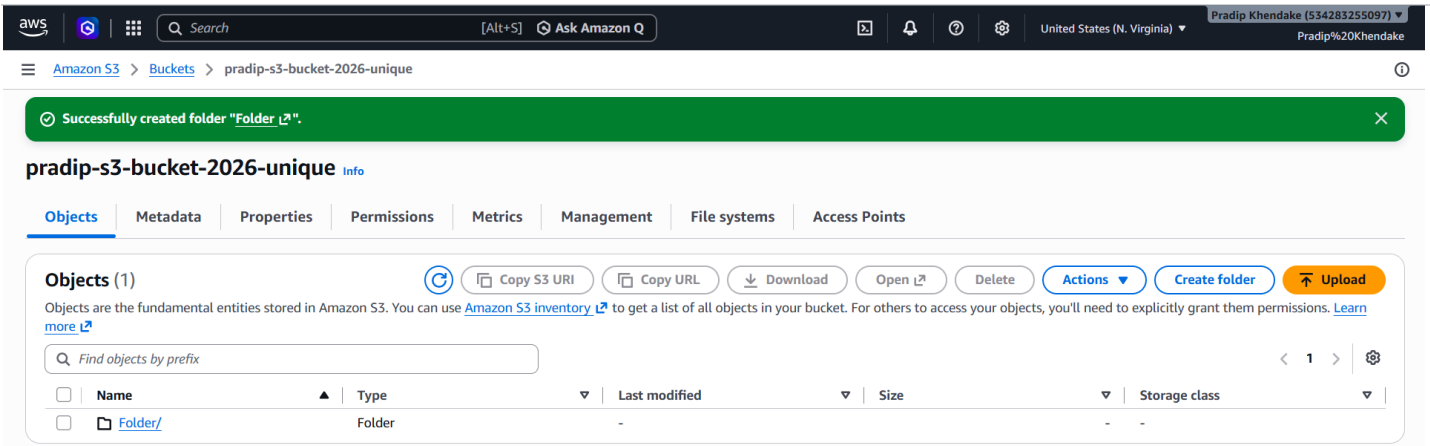


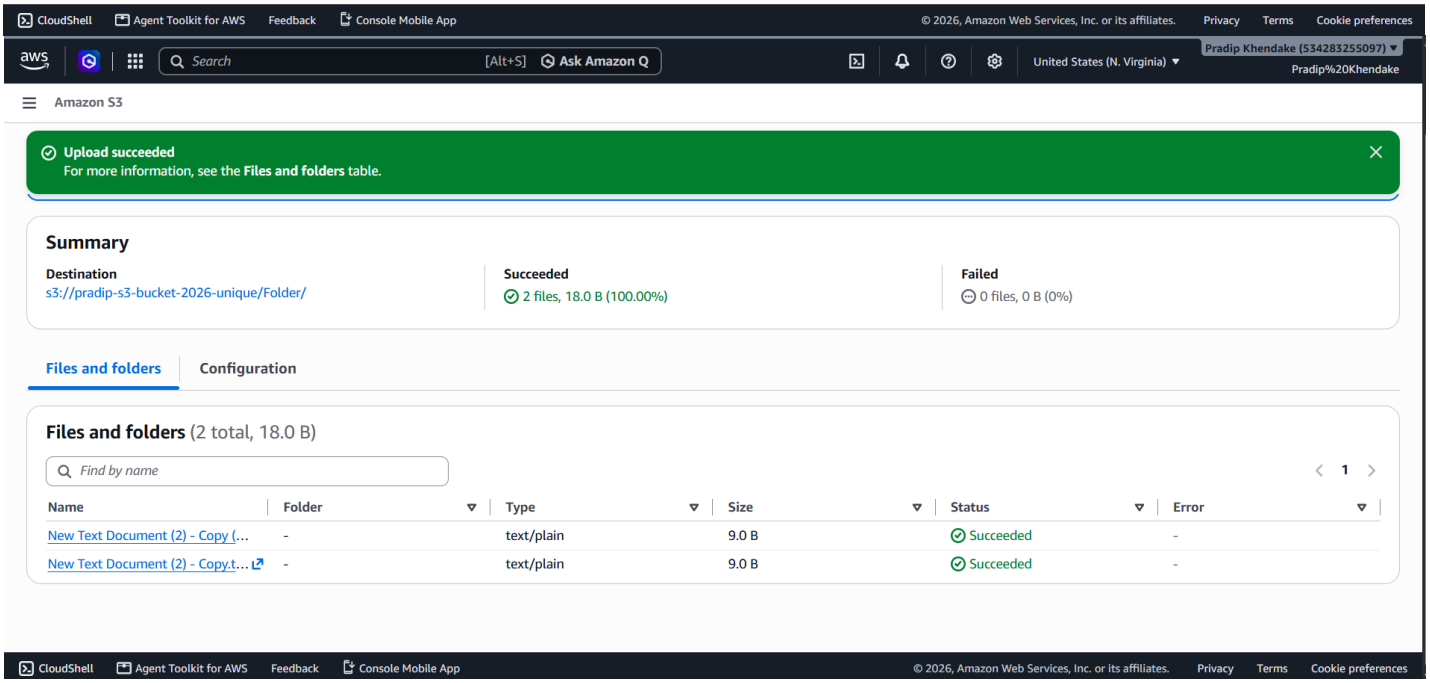
8. Download a file from an S3 bucket to your computer.

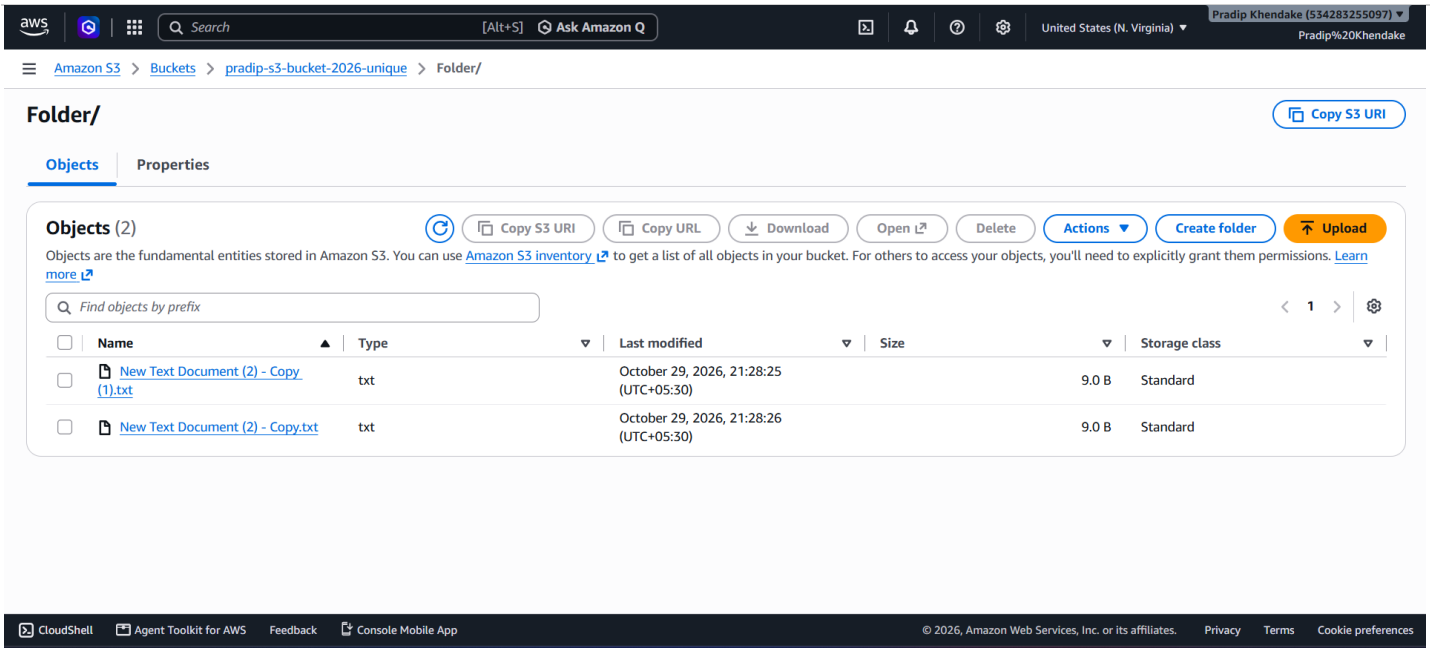


9. Create a folder in an S3 bucket and upload two files into it.

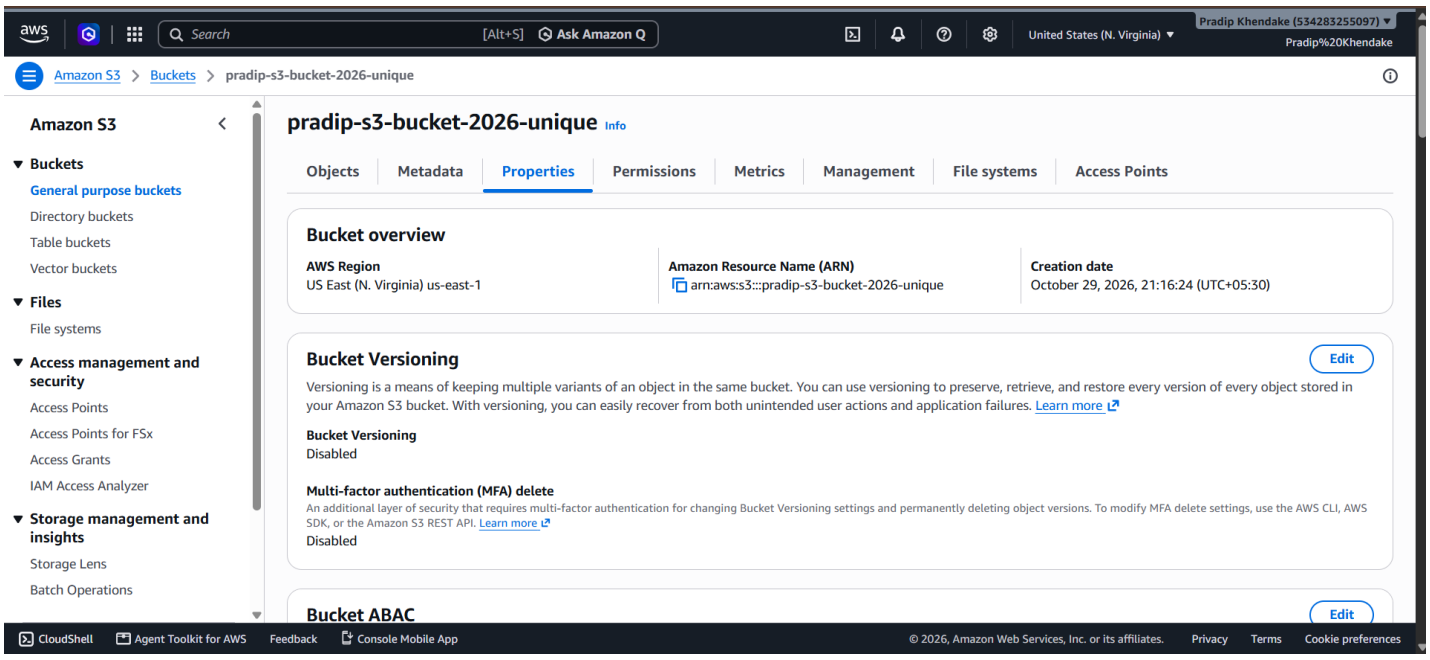


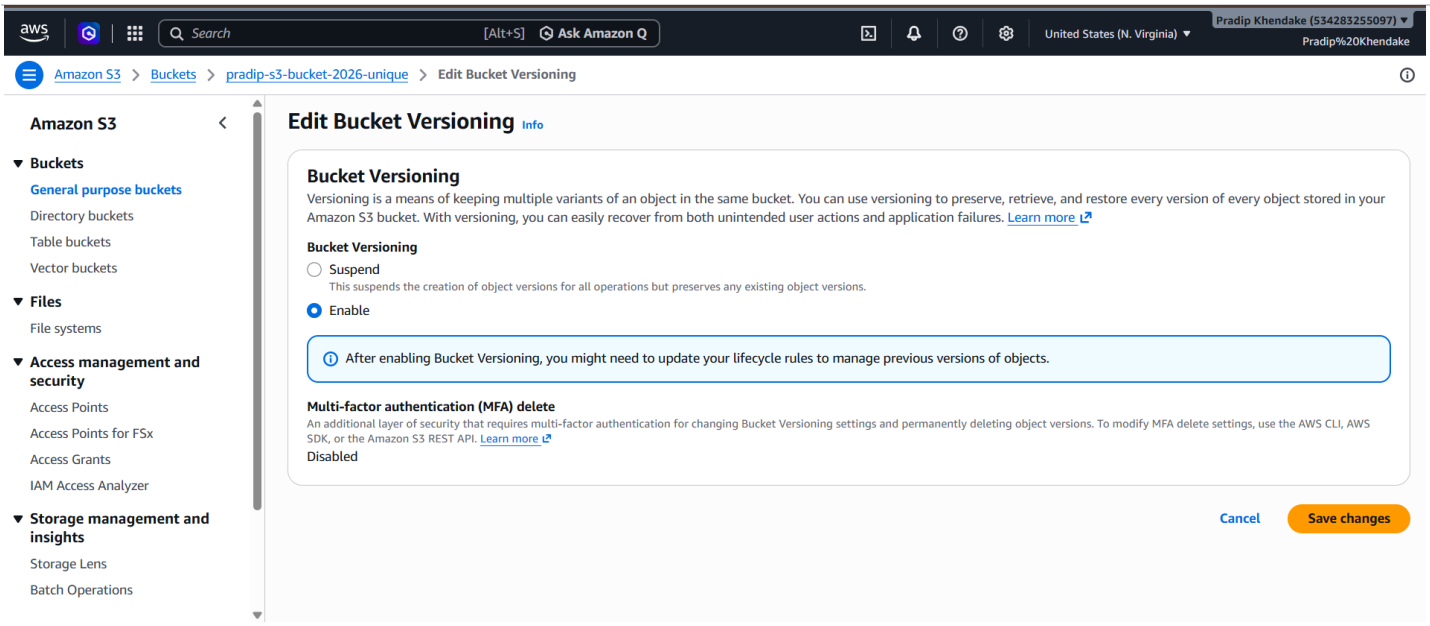

Search
[Alt+S]
Ask Amazon Q
United States (N. Virginia)
Pradip Khendake (534283255097)
Pradip%20Khendake
Amazon S3 > Buckets > pradip-s3-bucket-2026-unique
Successfully created folder "Folder".
pradip-s3-bucket-2026-unique Info
Objects
Metadata
Properties
Permissions
Metrics
Management
File systems
Access Points
Objects (1)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
Folder/
Folder

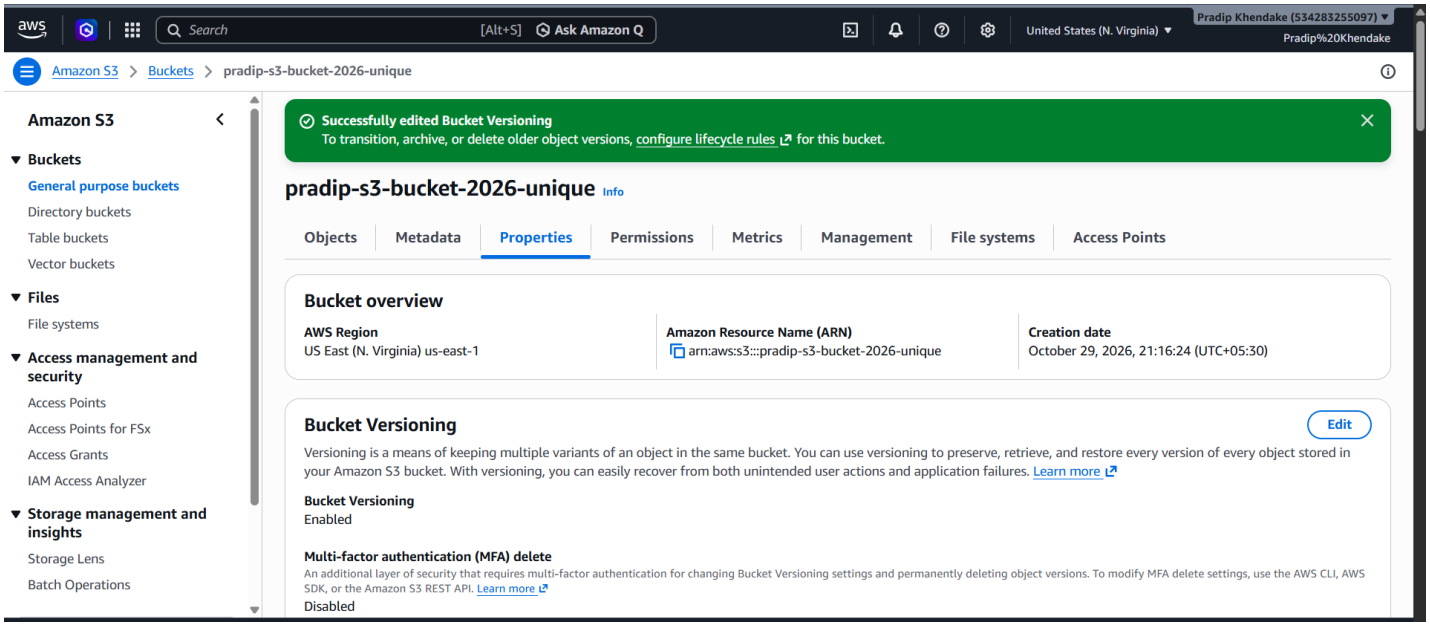

CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
Search
[Alt+S]
Ask Amazon Q
United States (N. Virginia)
Pradip Khendake (534283255097)
Pradip%20Khendake
Amazon S3
Upload succeeded
For more information, see the Files and folders table.
Summary
Destination
s3://pradip-s3-bucket-2026-unique/Folder/
Succeeded
2 files, 18.0 B (100.00%)
Failed
0 files, 0 B (0%)
Files and folders
Configuration
Files and folders (2 total, 18.0 B)
Find by name
Name
Folder
Type
Size
Status
Error
New Text Document (2) - Copy (...
text/plain
9.0 B
Succeeded
New Text Document (2) - Copy.t...
text/plain
9.0 B
Succeeded

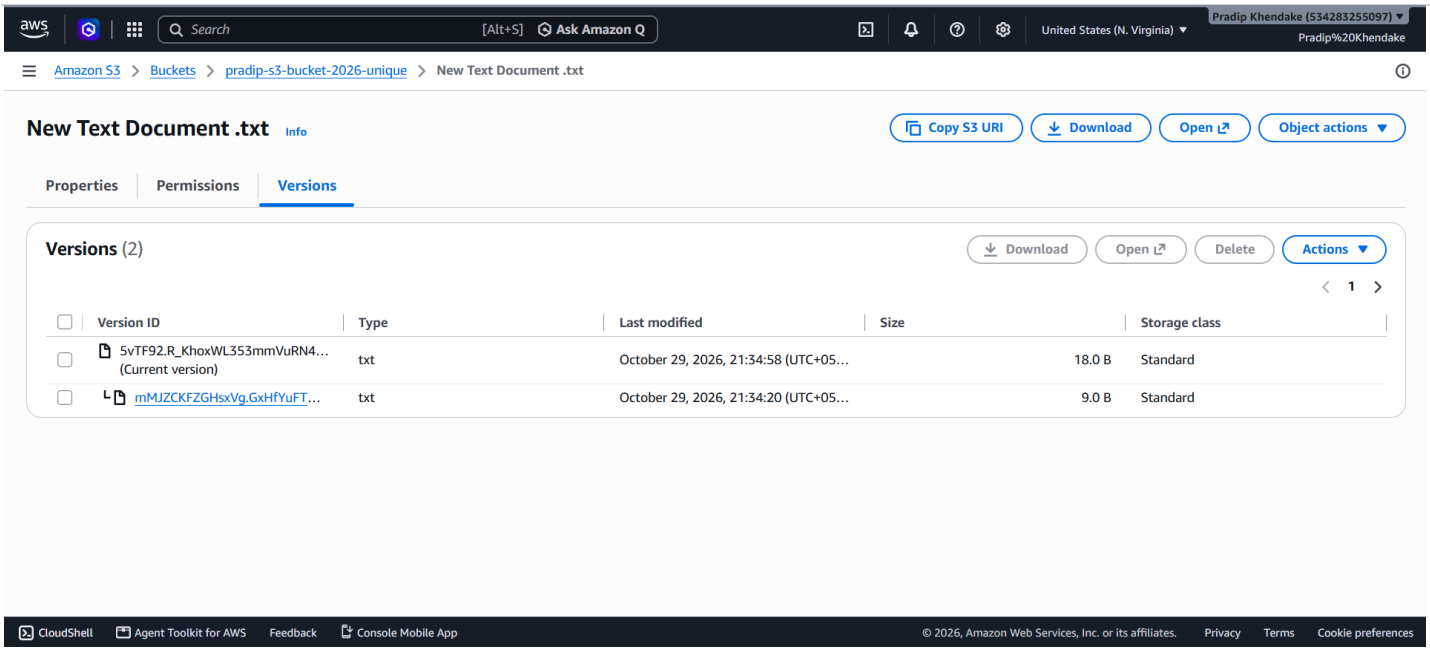


10. Enable versioning on an S3 bucket and upload multiple versions of the same file.

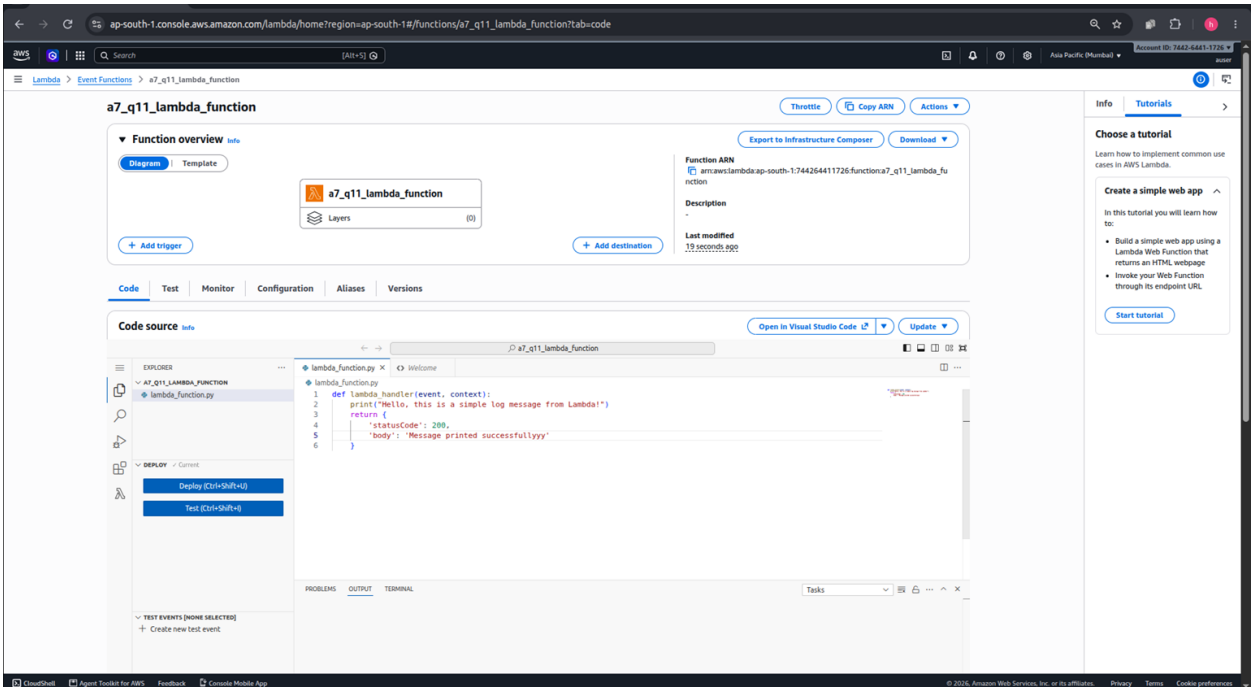


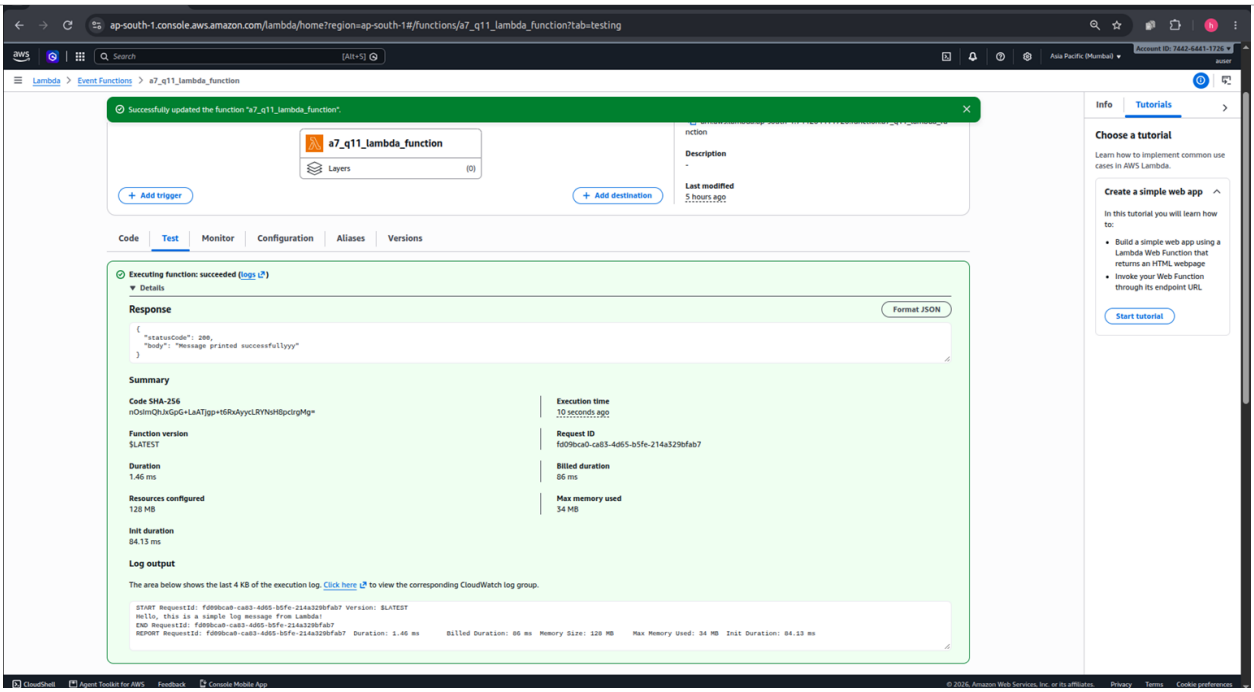

Search [Alt+S] Ask Amazon Q
United States (N. Virginia)
Pradip Khendake (534283255097)
Pradip%20Khendake
Amazon S3 > Buckets > pradip-s3-bucket-2026-unique > Edit Bucket Versioning
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Files
File systems
Access management and security
Access Points
Access Points for FSx
Access Grants
IAM Access Analyzer
Storage management and insights
Storage Lens
Batch Operations
Edit Bucket Versioning Info
Bucket Versioning
Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. Learn more
Bucket Versioning
Suspend
This suspends the creation of object versions for all operations but preserves any existing object versions.
Enable
After enabling Bucket Versioning, you might need to update your lifecycle rules to manage previous versions of objects.
Multi-factor authentication (MFA) delete
An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions. To modify MFA delete settings, use the AWS CLI, AWS SDK, or the Amazon S3 REST API. Learn more
Disabled
Cancel
Save changes


Search [Alt+S] Ask Amazon Q
United States (N. Virginia)
Pradip Khendake (534283255097)
Pradip%20Khendake
Amazon S3 > Buckets > pradip-s3-bucket-2026-unique
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Files
File systems
Access management and security
Access Points
Access Points for FSx
Access Grants
IAM Access Analyzer
Storage management and insights
Storage Lens
Batch Operations
Successfully edited Bucket Versioning
To transition, archive, or delete older object versions, configure lifecycle rules for this bucket.
pradip-s3-bucket-2026-unique Info
Objects
Metadata
Properties
Permissions
Metrics
Management
File systems
Access Points
Bucket overview
AWS Region
US East (N. Virginia) us-east-1
Amazon Resource Name (ARN)
arn:aws:s3:::pradip-s3-bucket-2026-unique
Creation date
October 29, 2026, 21:16:24 (UTC+05:30)
Bucket Versioning
Edit
Versioning is a means of keeping multiple variants of an object in the same bucket. You can use versioning to preserve, retrieve, and restore every version of every object stored in your Amazon S3 bucket. With versioning, you can easily recover from both unintended user actions and application failures. Learn more
Bucket Versioning
Enabled
Multi-factor authentication (MFA) delete
An additional layer of security that requires multi-factor authentication for changing Bucket Versioning settings and permanently deleting object versions. To modify MFA delete settings, use the AWS CLI, AWS SDK, or the Amazon S3 REST API. Learn more
Disabled

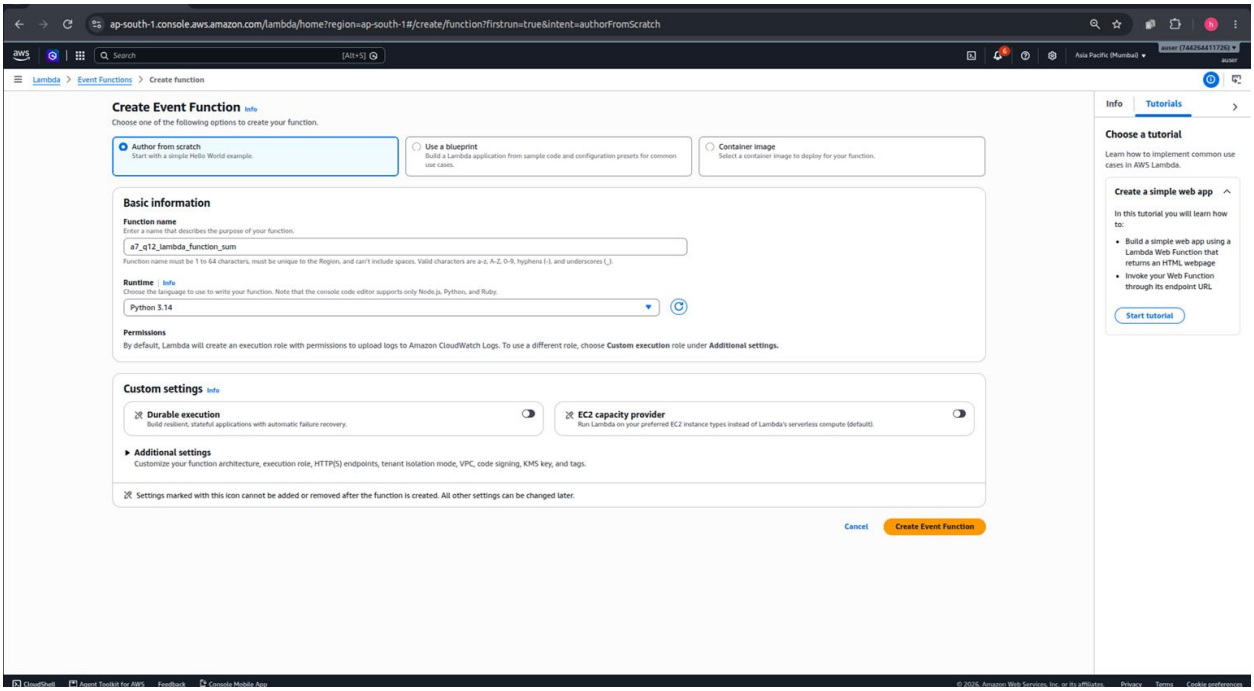


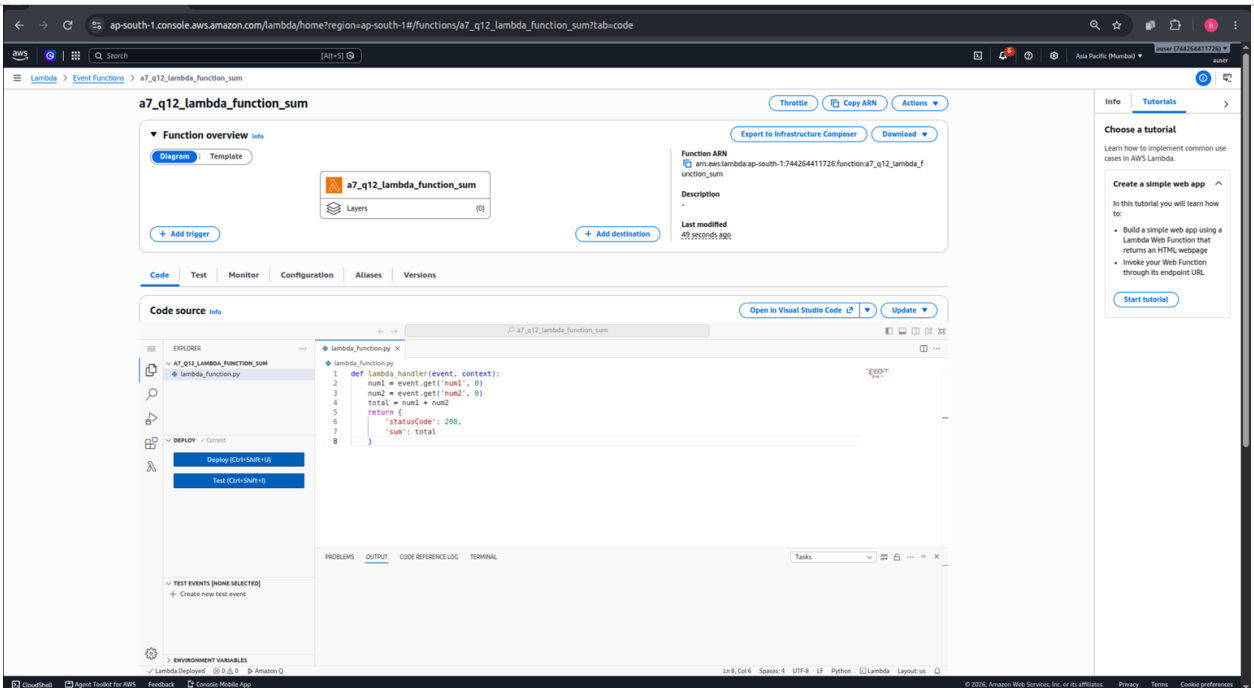
11. Create a Lambda function that prints a simple message.

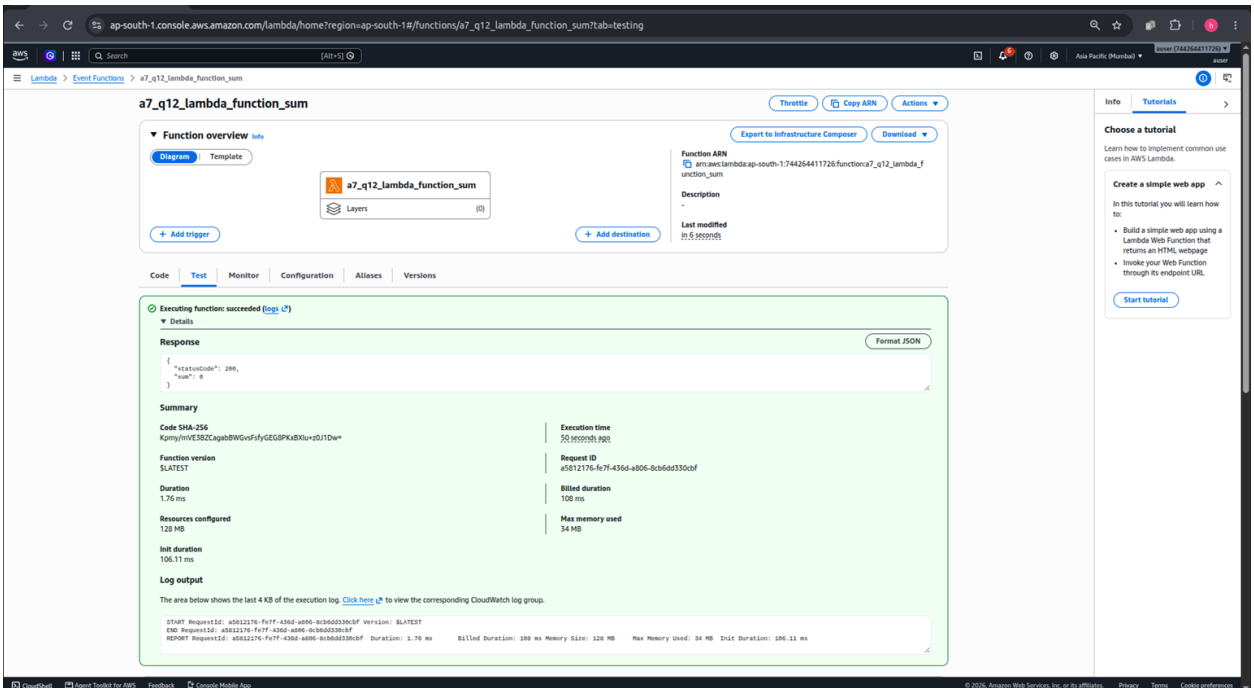




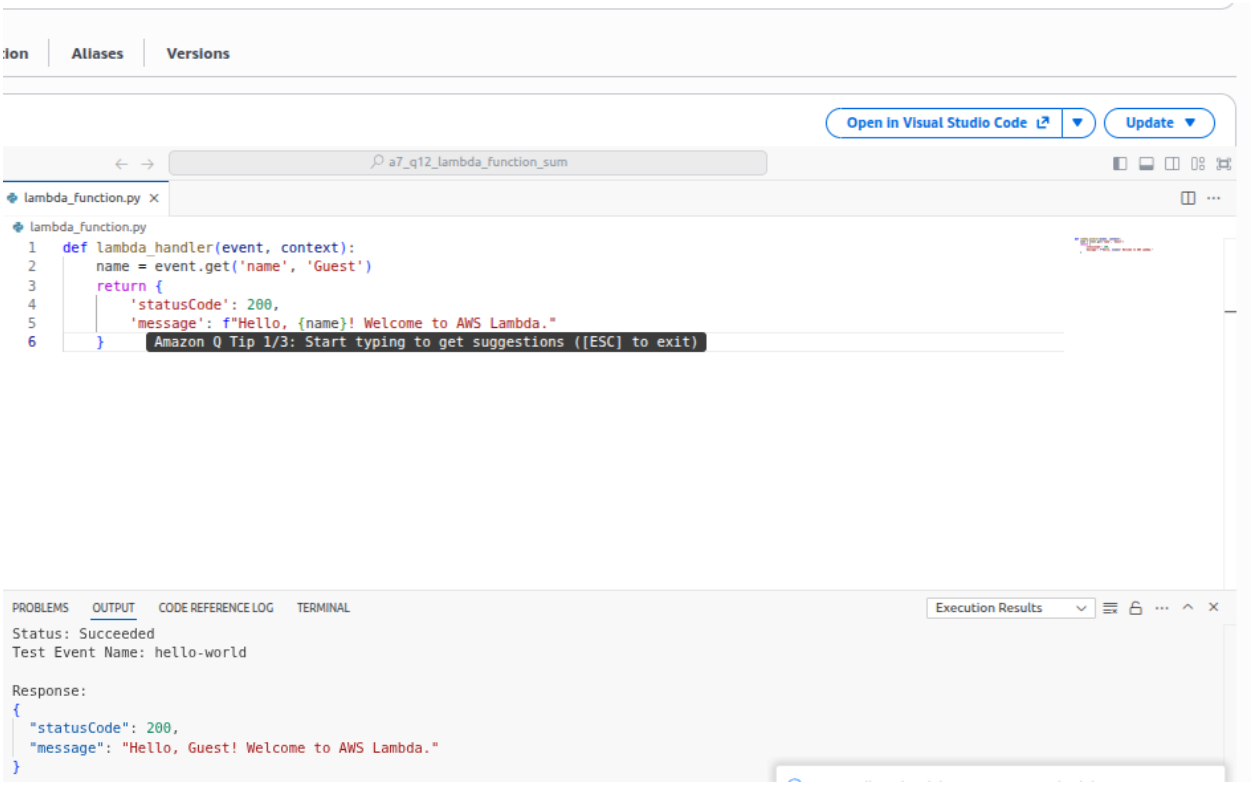
12. Create a Lambda function that accepts two numbers and returns their sum.

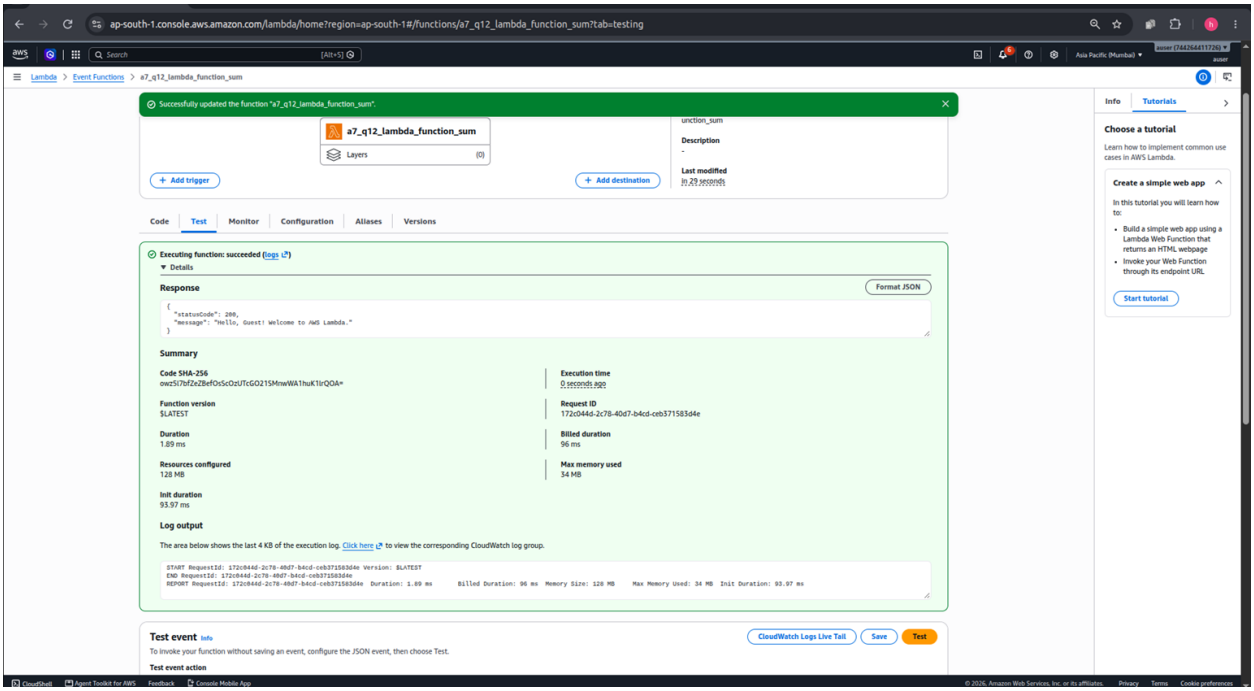



ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/functions/a7_q12_lambda_function_sum?tab=code
a7_q12_lambda_function_sum
Throttle
Copy ARN
Actions
Function overview
Export to Infrastructure Composer
Download
Diagram
Template
a7_q12_lambda_function_sum
Layers
(0)
Add trigger
Add destination
Function ARN
arn:aws:lambda:ap-south-1:744264411726:function:a7_q12_lambda_function_sum
Description
Last modified
49 seconds ago
Code
Test
Monitor
Configuration
Aliases
Versions
Code source
Open in Visual Studio Code
Update
lambda_function.py
def lambda_handler(event, context):
num1 = event.get('num1', 0)
num2 = event.get('num2', 0)
total = num1 + num2
return {
'statusCode': 200,
'sum': total
}
Deploy (Ctrl+Shift+U)
Test (Ctrl+Shift+I)
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app using a Lambda Web Function that returns an HTML webpage
Invoke your Web Function through its endpoint URL
Start tutorial

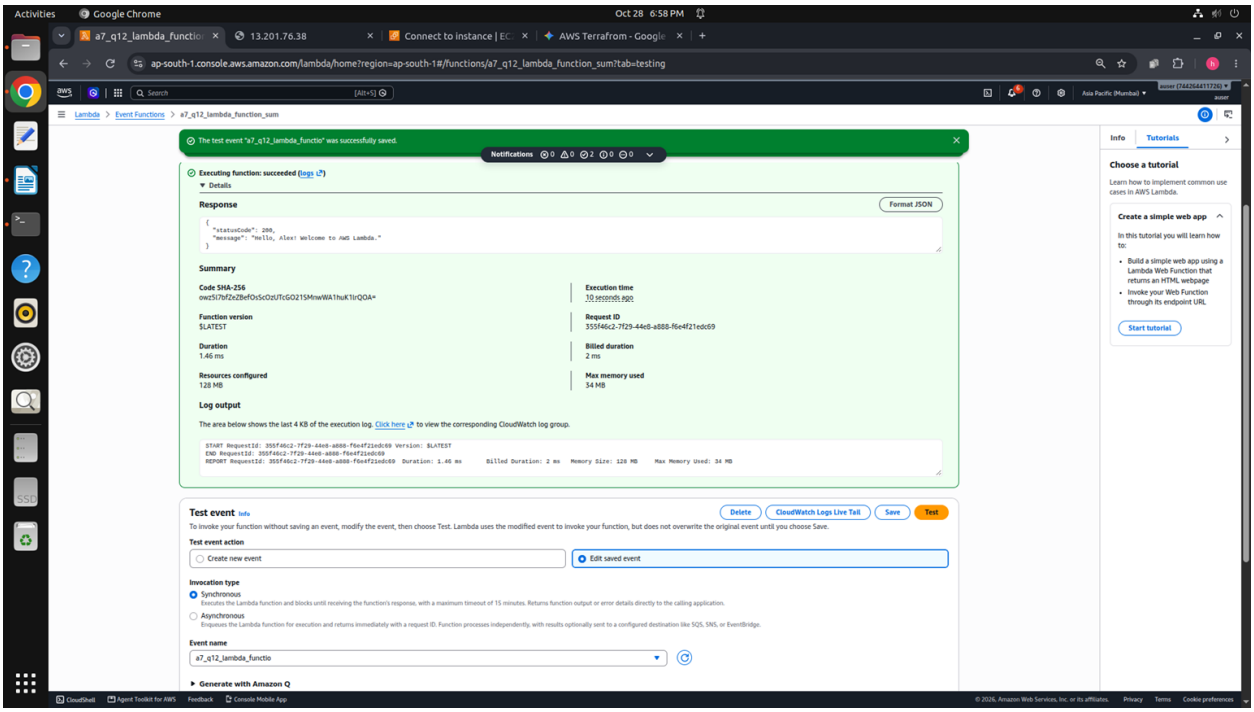

ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/functions/a7_q12_lambda_function_sum?tab=testing
a7_q12_lambda_function_sum
Throttle
Copy ARN
Actions
Function overview
Export to Infrastructure Composer
Download
Diagram
Template
a7_q12_lambda_function_sum
Layers
(0)
Add trigger
Add destination
Function ARN
arn:aws:lambda:ap-south-1:744264411726:function:a7_q12_lambda_function_sum
Description
Last modified
in 6 seconds
Code
Test
Monitor
Configuration
Aliases
Versions
Executing function: succeeded (logs)
Details
Response
Format JSON
{
"statusCode": 200,
"sum": 0
}
Summary
Code SHA-256
Kpmy/mVE3BZCagabBWGvsFsfyGEGBPKxBXlu+z0J1Dw=
Execution time
50 seconds ago
Function version
$LATEST
Request ID
a5812176-fe7f-436d-a806-8cb6dd330cbf
Duration
1.76 ms
Billed duration
108 ms
Resources configured
128 MB
Max memory used
34 MB
Init duration
106.11 ms
Log output
The area below shows the last 4 KB of the execution log. Click here to view the corresponding CloudWatch log group.
START RequestId: a5812176-fe7f-436d-a806-8cb6dd330cbf Version: $LATEST
END RequestId: a5812176-fe7f-436d-a806-8cb6dd330cbf
REPORT RequestId: a5812176-fe7f-436d-a806-8cb6dd330cbf Duration: 1.76 ms Billed Duration: 108 ms Memory Size: 128 MB Max Memory Used: 34 MB Init Duration: 106.11 ms
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app using a Lambda Web Function that returns an HTML webpage
Invoke your Web Function through its endpoint URL
Start tutorial

13. Create a Lambda function that accepts a name as input and returns a greeting.

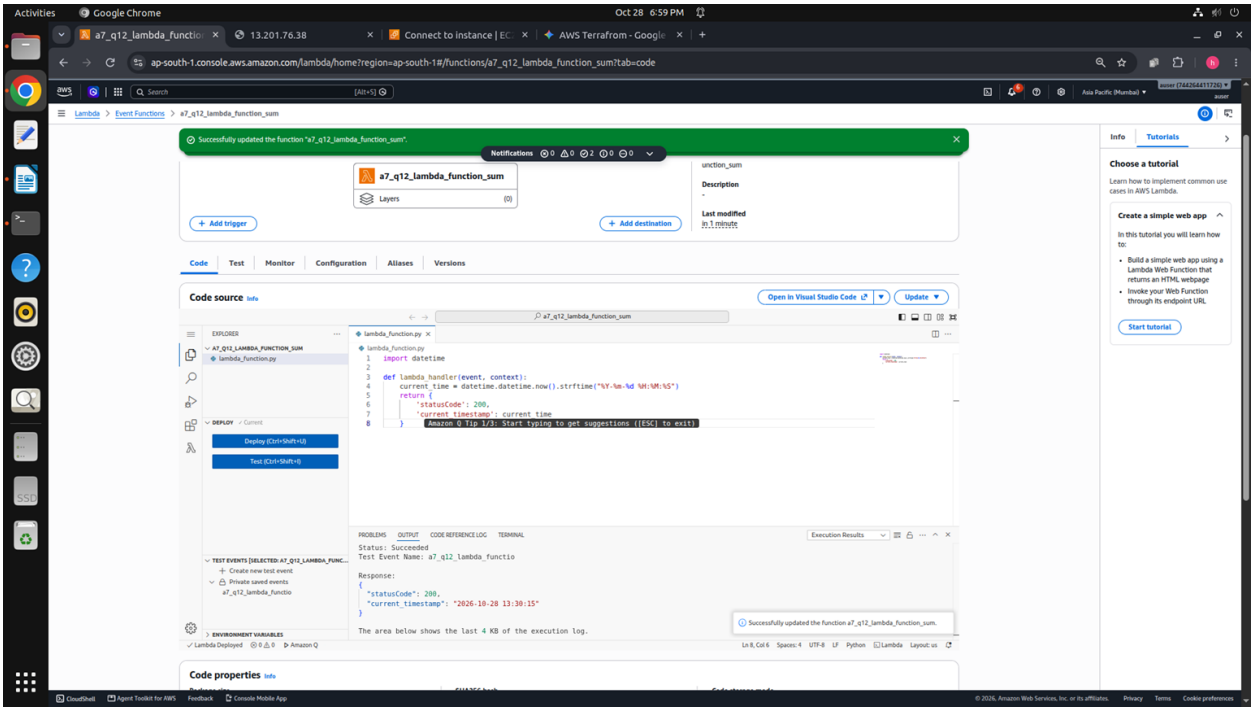




14. Test a Lambda function using different JSON inputs and observe the outputs.

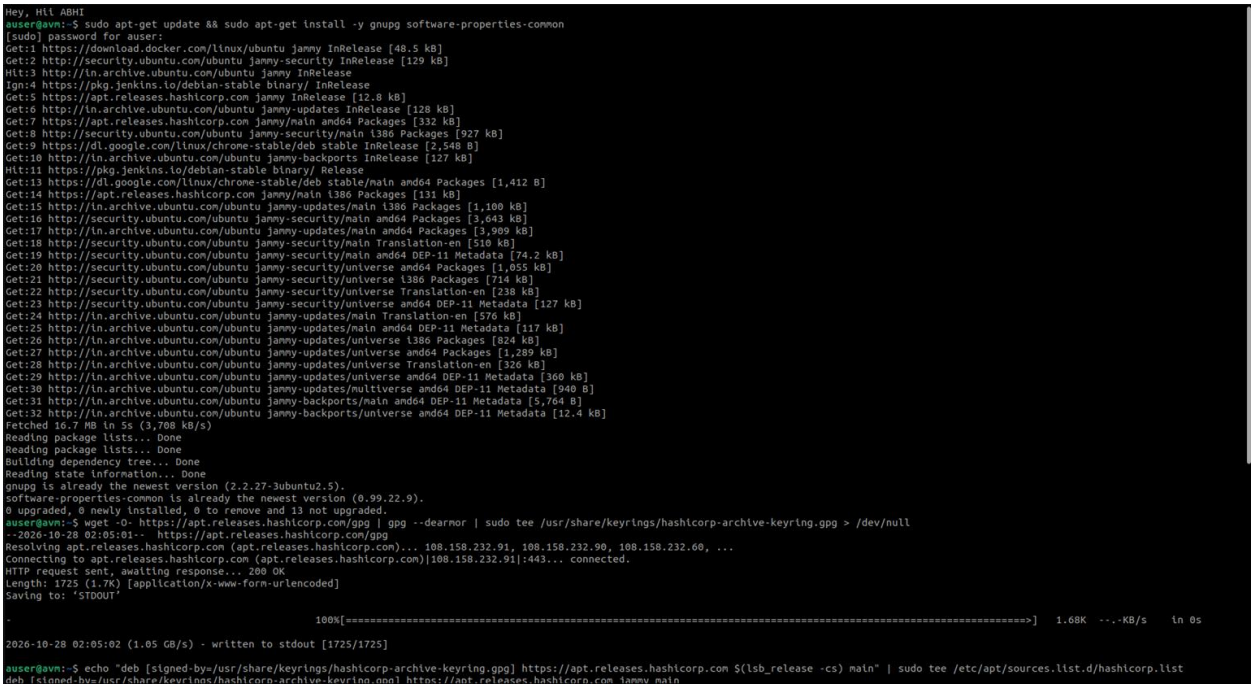


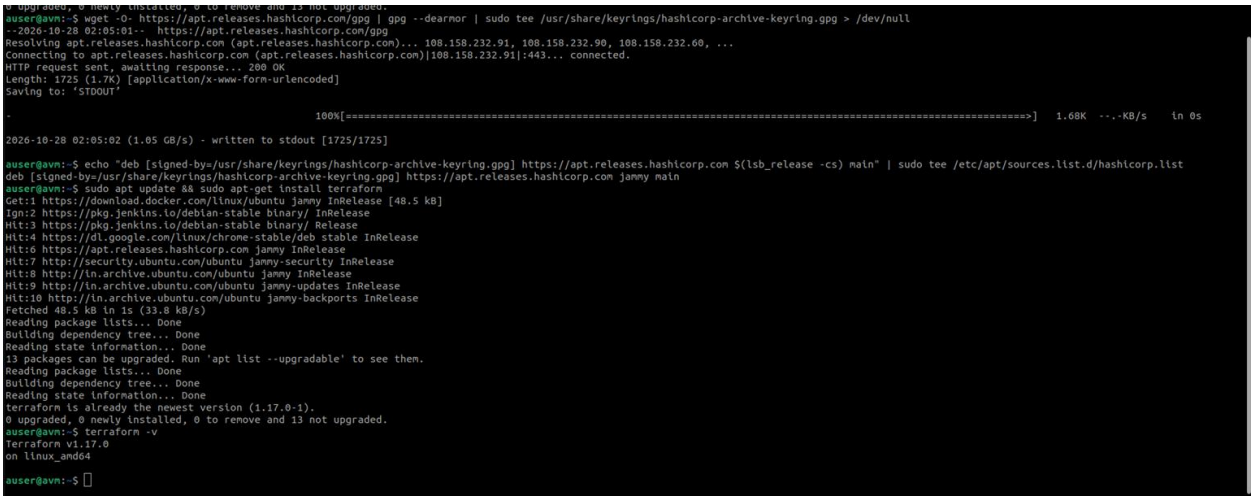
15. Create a Lambda function that returns the current date and time.



Terraform

1. Install Terraform and verify its version.





2. Write a Terraform configuration using the local provider to create a text file containing a welcome message.

```
auser@avm:~$ mkdir terraform_welcome
auser@avm:~$ cd terraform_welcome/
auser@avm:~/terraform_welcome$ nano main.tf
auser@avm:~/terraform_welcome$ terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.0"...
- Installing hashicorp/local v2.9.1...
- Installed hashicorp/local v2.9.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
auser@avm:~/terraform_welcome$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.welcome will be created
  + resource "local_file" "welcome" {
      + content              = "Welcome to terraform Learning"
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "welcome.txt"
      + id                   = (known after apply)
    }
```

```
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
auser@avm:~/terraform_welcome$ terrafrom apply -auto-approve
Command 'terrafron' not found, did you mean:
  command 'terraform' from snap terraform (1.16.7)
See 'snap info <snapname>' for additional versions.
auser@avm:~/terraform_welcome$ terraform apply -auto-approve

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

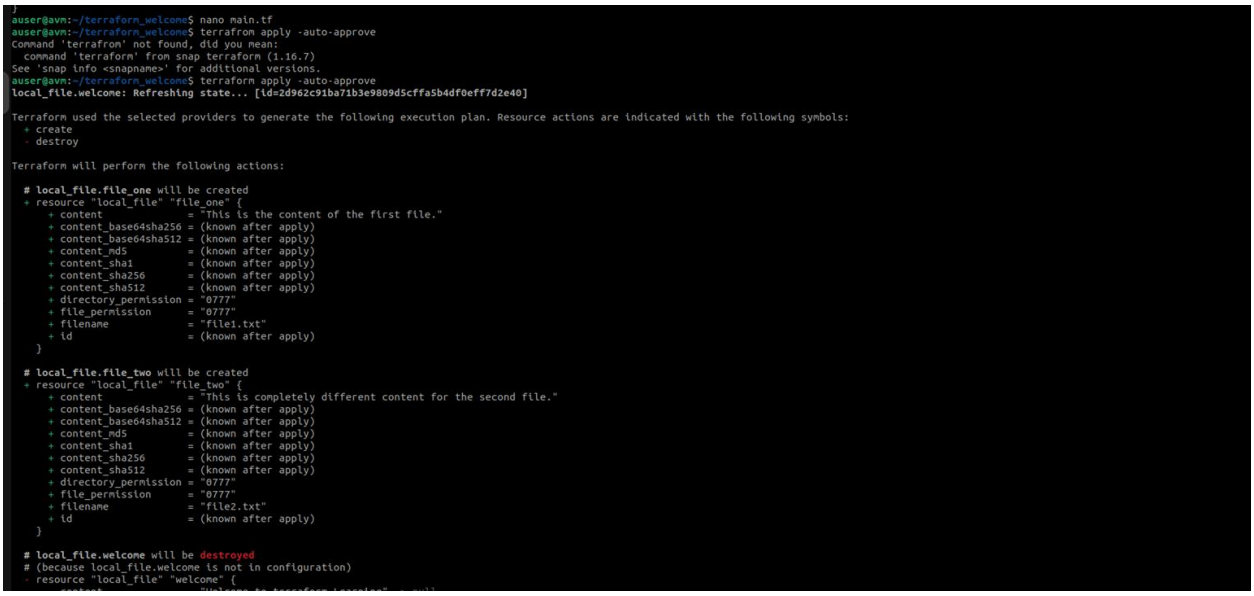
  # local_file.welcome will be created
  + resource "local_file" "welcome" {
      + content              = "Welcome to terraform Learning"
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "welcome.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
local_file.welcome: Creating...
local_file.welcome: Creation complete after 0s [id=2d962c91ba71b3e9809d5cffa5b4df0eff7d2e40]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
auser@avm:~/terraform_welcome$ cat welcome.txt
Welcome to terraform Learningauser@avm:~/terraform_welcome$ cat main.tf
terraform{
        required_providers{
                local = {
                        source = "hashicorp/local"
                        version = "~>2.0"
                }
        }
}
provider "local" {}

resource "local_file" "welcome" {
        filename = "welcome.txt"
        content  = "Welcome to terraform Learning"
}
auser@avm:~/terraform_welcome$
```

3. Modify the Terraform configuration to create two text files with different contents.





4. Use Terraform variables to specify the filename and file contents.

```
local_file.file_one: Destruction complete after 0s
local_file.file_two: Destruction complete after 0s

Destroy complete! Resources: 2 destroyed.
auser@avm:~/terraform_welcome$ cd
auser@avm:~$ mkdir terraform_vars_lab
auser@avm:~$ cd terraform_vars_lab
auser@avm:~/terraform_vars_lab$ nano variables.tf
auser@avm:~/terraform_vars_lab$ nano main.tf
auser@avm:~/terraform_vars_lab$ terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.0"...
- Installing hashicorp/local v2.9.1...
- Installed hashicorp/local v2.9.1 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
auser@avm:~/terraform_vars_lab$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.example will be created
  + resource "local_file" "example" {
      + content              = "Hello, this content came from a Terraform variable!"
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "variable_output.txt"
      + id                   = (known after apply)
    }
```

```
Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
auser@avm:~/terraform_vars_lab$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.example will be created
  + resource "local_file" "example" {
      + content              = "Hello, this content came from a Terraform variable!"
      + content_base64sha256 = (known after apply)
      + content_base64sha512 = (known after apply)
      + content_md5          = (known after apply)
      + content_sha1         = (known after apply)
      + content_sha256       = (known after apply)
      + content_sha512       = (known after apply)
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "variable_output.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

local_file.example: Creating...
local_file.example: Creation complete after 0s [id=811338face3f57356b03f025122014f1e425b594]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
auser@avm:~/terraform_vars_lab$ ls
main.tf  terraform.tfstate  variable_output.txt  variables.tf
auser@avm:~/terraform_vars_lab$ cat variable_output.txt
Hello, this content came from a Terraform variable!auser@avm:~/terraform_vars_lab$ cat main.tf
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.0"
    }
  }
}

provider "local" {}
```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
auser@avm:~/terraform_vars_lab$ ls
main.tf  terraform.tfstate  variable_output.txt  variables.tf
auser@avm:~/terraform_vars_lab$ cat variable_output.txt
Hello, this content came from a Terraform variable!auser@avm:~/terraform_vars_lab$ cat main.tf
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.0"
    }
  }
}

provider "local" {}

resource "local_file" "example" {
  filename = var.filename
  content  = var.file_content
}
auser@avm:~/terraform_vars_lab$ cat variables.tf
variable "filename" {
  description = "The target name of the file"
  type        = string
  default     = "variable_output.txt"
}

variable "file_content" {
  description = "The text content to write inside the file"
  type        = string
  default     = "Hello, this content came from a Terraform variable!"
}
auser@avm:~/terraform_vars_lab$
```

5. Use Terraform outputs to display the names and contents of the files created by the configuration.

```
}
auser@avm:~/terraform_vars_lab$ nano outputs.tf
auser@avm:~/terraform_vars_lab$ terraform apply -auto-approve
local_file.example: Refreshing state... [id=811338face3f57356b03f025122014f1e425b594]

Changes to Outputs:
  + created_file_content = "Hello, this content came from a Terraform variable!"
  + created_file_name    = "variable_output.txt"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

created_file_content = "Hello, this content came from a Terraform variable!"
created_file_name = "variable_output.txt"
auser@avm:~/terraform_vars_lab$ cat outputs.tf
output "created_file_name" {
  description = "The name of the generated file"
  value       = local_file.example.filename
}

output "created_file_content" {
  description = "The content written inside the generated file"
  value       = local_file.example.content
}
auser@avm:~/terraform_vars_lab$
```